*A PROJECT REPORT ON*

# KRISHIMITRA: AI-POWERED AGRICULTURAL DECISION SUPPORT WEB APPLICATION

*Submitted in partial fulfillment for the award of the degree of*

## BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING

**TEAM PROJECT**

**KAAVYA GUPTA (23BCE1986)**

**SHIVOM SHRIVASTAVA (23BCE1983)**

**ANNALAMUNI PRAVEEN (23BCE1648)**

**SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**

**VELLORE INSTITUTE OF TECHNOLOGY · CHENNAI**

OCTOBER 2026

# KRISHIMITRA: AI-POWERED AGRICULTURAL DECISION SUPPORT WEB APPLICATION

**PROJECT REPORT**

*Submitted in partial fulfillment for the award of the degree of*

# BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING

*by*

**KAAVYA GUPTA (23BCE1986)**

**SHIVOM SHRIVASTAVA (23BCE1983)**

**ANNALAMUNI PRAVEEN (23BCE1648)**

Project Guide: Dr. Premlatha

## SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

**VELLORE INSTITUTE OF TECHNOLOGY · CHENNAI**

OCTOBER 2026

## DECLARATION

We hereby declare that the project report entitled “KrishiMitra: AI-Powered Agricultural Decision Support Web Application” is an account of the project work carried out by us under the guidance of Dr. Premlatha, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

We further declare that the work reported in this project has not been submitted, either in part or in full, for the award of any other degree or diploma at this or any other institution or university. All external sources used in the report are acknowledged through in-text citations and the reference list.

Place: Chennai

Date: _____

Signatures of the candidates:

Kaavya Gupta (23BCE1986): _____

Shivom Shrivastava (23BCE1983): _____

Annalamuni Praveen (23BCE1648): _____

## CERTIFICATE

This is to certify that the project report entitled “KrishiMitra: AI-Powered Agricultural Decision Support Web Application” is prepared and submitted by Kaavya Gupta (23BCE1986), Shivom Shrivastava (23BCE1983), and Annalamuni Praveen (23BCE1648) to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering. The project work was carried out under my guidance.

The report is submitted for academic evaluation. The signatures, dates, and institutional seal below are intentionally left blank for completion by the relevant authorities.

Signature of the Guide: _____

Name: Dr. Premlatha

Date: _____

Signature of Examiner 1: _____ Signature of Examiner 2: _____

Name: _____ Name: _____

Date: _____ Date: _____

Approved by the Head of Department: _____

Name: _____

Date: _____

(Seal of the School)

## ABSTRACT

KrishiMitra is a browser-based agricultural decision-support prototype that brings together crop-health image screening, location-based crop planning, and a local agronomy text advisor. The project is implemented as a React, TypeScript and Vite frontend backed by FastAPI, Python modules and PostgreSQL. This report documents the current implementation and distinguishes live application behavior from offline experiments and literature-reported scores.

Module 1 validates an uploaded leaf image and processes it with a MobileNetV2 classifier covering ten supported labels across Tomato, Potato, Maize and Grape. It returns the highest-scoring class and marks the result uncertain when confidence is below 75 percent or when the selected crop conflicts with the model's crop result. Scan metadata and an image hash are stored; the uploaded photograph is not retained with that metadata. The saved PlantVillage-derived test report records 98.68 percent

accuracy, 98.91 percent macro precision, 98.37 percent macro recall and 98.63 percent macro F1 on 1,591 cases. The source images are missing from the current project folder, so these are saved results and were not independently rerun for this report.

Module 2 contains a transparent rule-based planner and experimental Random Forest and XGBoost yield regressors. The planner combines a selected location and season with OpenLandMap soil estimates, Open-Meteo weather, FAO crop ranges, and published national yield, minimum support price and A2+FL cost benchmarks. Eligible crops are ranked by benchmark surplus per hectare. This is not a farm-specific profit estimate. Separately, an offline chronological evaluation on 10,011 district-yield records from 2013–2017 measured Random Forest MAE 676.89 kg/ha, RMSE 1,071.50 kg/ha and $R^2$ 0.116; XGBoost MAE 683.10 kg/ha, RMSE 1,080.97 kg/ha and $R^2$ 0.100. These results evaluate historical yield regression, not crop-choice accuracy or field profit. [1]

Module 3 currently uses deterministic query expansion and PostgreSQL full-text search to return stored, curated agronomy passages with source links and page or section details when available. An earlier iteration considered Gemini-based generation and RAG; the current text advisor does not use a generative model to compose answers. The attempted retrieval benchmark did not run because the configured database lacked a current search-vector column. No retrieval-quality score is claimed. Comparisons with CNN, Random Forest and XGBoost research are included as context, with explicit warnings about different data, targets, splits and metrics. [2]

Keywords: agricultural decision support, MobileNetV2, plant disease screening, crop planning, Random Forest, XGBoost, yield regression, PostgreSQL full-text retrieval, evaluation

## ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Dr. Premlatha, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for her guidance and feedback during the development and documentation of KrishiMitra.

Her support helped us examine the project's modules as a connected system and to state the limits of its outputs clearly.

We thank the faculty and staff of the School of Computer Science and Engineering for the academic environment and resources that supported this work. We also acknowledge the maintainers of the public datasets, software libraries, public environmental data services and research publications cited in this report. Their documentation and data made it possible to develop the prototype and conduct the recorded offline evaluations.

We are grateful to our families and peers for their encouragement throughout the project. Any remaining errors in this report or in the prototype are our responsibility.

Place: Chennai

Date: _____

Kaavya Gupta (23BCE1986) Shivom Shrivastava (23BCE1983) Annalamuni Praveen (23BCE1648)

# TABLE OF CONTENTS


| | |
|---|---|
| INTRODUCTION | 12 |
| Agricultural decision-support context | 12 |
| Problem statement | 12 |
| Aim and objectives | 13 |
| Scope and boundaries | 13 |
| Users and use scenarios | 14 |
| Report organization | 14 |
| BACKGROUND AND LITERATURE REVIEW | 15 |
| Plant disease classification from leaf images | 15 |
| MobileNetV2 and transfer learning | 16 |
| Controlled data and field domain shift | 16 |
| Crop planning as a multi-criteria decision | 17 |
| Random Forest for tabular regression | 18 |
| XGBoost for tabular regression | 18 |
| Historical yield forecasting research | 19 |
| Text retrieval, RAG and citation-grounded answers | 19 |
| Research gap and project position | 20 |
| REQUIREMENTS AND SYSTEM ARCHITECTURE | 21 |
| MODULE 1: CROP HEALTH SCREENING | 24 |
| MODULE 2: SOIL AND CROP PLANNING | 30 |
| MODULE 3: LOCAL AGRONOMY ADVISOR | 37 |
| IMPLEMENTATION AND DATA FLOW | 41 |
| TESTING AND EVALUATION | 45 |
| RESULTS, LITERATURE COMPARISON AND DISCUSSION | 50 |
| CONCLUSION AND FUTURE WORK | 53 |
| REFERENCES | 56 |
| APPENDICES | 57 |

# LIST OF TABLES

Table 4.2. Saved MobileNetV2 evaluation report 28
Table 4.3. Stored classwise scores 28
Table 4.4. MobileNetV2 research comparison 29
Table 5.1. Offline yield-regression evaluation 35
Table 5.2. Separate live UI examples 35
Table 5.3. Research comparison 36
Table 8.1. Stored Module 1 aggregate metrics 47
Table 8.2. Saved final-run temporal evaluation 48

# LIST OF FIGURES

Figure 3.1. Overall KrishiMitra system architecture 21
Figure 3.2. Frontend home screenshot placeholder 24
Figure 4.1. Module 1 image screening workflow 25
Figure 4.2. Saved per-class MobileNetV2 F1 scores 29
Figure 4.3. Module 1 literature comparison graph 29
Figure 4.4. Module 1 prediction screenshot placeholder 29
Figure 4.5. Module 1 uncertainty screenshot placeholder 29
Figure 5.1. UI benchmark-surplus examples 36
Figure 5.2. Module 2 offline metrics graph 36
Figure 5.3. Simple planner screenshot placeholder 37
Figure 5.4. Random Forest UI screenshot placeholder 37
Figure 5.5. XGBoost UI screenshot placeholder 37
Figure 5.6. Offline evaluation screenshot placeholder 37
Figure 6.1. Current Module 3 retrieval architecture 38
Figure 6.2. Module 3 retrieved answer screenshot placeholder 41
Figure 8.1. Evaluation targets by module 46

# LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

| Abbreviation / symbol | Meaning |
|---|---|
| API | Application Programming Interface |
| A2+FL | Paid-out cost plus imputed value of family labour |
| CNN | Convolutional Neural Network |
| DLD | District-level historical yield dataset used by experimental model |
| FAO | Food and Agriculture Organization of the United Nations |
| F1 | Harmonic mean of precision and recall |
| GPS | Global Positioning System |
| HTTP | Hypertext Transfer Protocol |
| JSON | JavaScript Object Notation |
| MAE | Mean Absolute Error |
| ML | Machine Learning |
| MobileNetV2 | Mobile-oriented convolutional neural network architecture |
| MSP | Minimum Support Price |
| nDCG | Normalized Discounted Cumulative Gain |
| NPK | Nitrogen, phosphorus and potassium |
| pH | Measure of acidity/alkalinity |
| P/R | Precision / recall |
| $R^2$ | Coefficient of determination |
| RAG | Retrieval-Augmented Generation |
| RF | Random Forest |
| RMSE | Root Mean Squared Error |
| SHC | Soil Health Card |
| SQL | Structured Query Language |
| TSV | Text Search Vector |
| UI | User Interface |
| Vite | Frontend build and development tool |
| XGBoost | Extreme Gradient Boosting |

CHAPTER 1

# INTRODUCTION

## 1.1 AGRICULTURAL DECISION-SUPPORT CONTEXT

Agricultural decisions are made under local and changing conditions. A crop that appears attractive in a national yield or price table may not suit a particular field, sowing window, soil, water supply, variety or market. A leaf image may resemble a known disease class but still be misleading when the image is blurred, the crop is unsupported, or symptoms are caused by nutrient stress or weather. Agronomy questions also require source-aware advice because recommendations can vary by region, season, crop stage and current regulations. These constraints motivate a decision-support design that presents evidence and conditions rather than one unqualified answer.

KrishiMitra organizes three different kinds of assistance in a single interface. The crop-health module is an image classifier. The crop-planning module combines public environmental data with agronomic screening rules and economic benchmarks; its experimental machine-learning tab forecasts district-level historical yield for comparison. The text advisor searches stored source passages. These modules share a browser and backend, but they do not share one prediction model or one definition of accuracy. The report therefore explains each module separately and distinguishes predictions, rules, retrieved source text, benchmarks and software tests.

## 1.2 PROBLEM STATEMENT

The project addresses the practical difficulty of exploring crop-health and crop-planning information through a single accessible web workflow. The software must accept user inputs safely, explain how a crop shortlist is assembled, show when a disease classification is uncertain, and return agronomy passages with traceable source information. It must also make comparisons between a transparent simple planning engine and experimental Random Forest and XGBoost models possible without presenting model output as established farm advice.

The present implementation does not solve the full agricultural decision problem. It does not collect field-confirmed disease images at scale, estimate the farmer’s complete cost structure, incorporate a complete district crop calendar, predict local

sale prices, or validate recommendations through controlled farm trials. The Module 2 yield dataset is historical and district-level, while the user-facing planning inputs describe a planned field. The gap between those two data contexts is a core limitation that shapes the claims made in this report.

## 1.3 AIM AND OBJECTIVES

The aim is to implement and document a modular agricultural decision-support web application whose outputs are transparent enough to inspect and cautious enough to communicate limitations. The objectives below are mapped to actual project evidence rather than only intended future behavior.

| Objective | Implementation evidence | Current status |
|---|---|---|
| Provide leaf-health screening for supported crops. | Image checks, MobileNetV2 ten-class inference, confidence and crop-conflict flag. | Implemented; saved evaluation exists, images unavailable for rerun. |
| Offer a location-aware crop planning workflow. | Public soil/weather lookups, agronomic screens, benchmark ranking. | Implemented; software tests and public-service smoke checks recorded. |
| Compare RF and XGBoost on the same historical-yield test set. | Offline time-based train/test experiment and common metrics. | Completed; modest $R^2$ values and known split limitations. |
| Return source-backed agronomy text. | Rule expansion, PostgreSQL full-text search, passages and citations. | Implemented in code; quality benchmark blocked by local DB schema. |
| Produce evidence for academic reporting. | Evaluation matrix, data provenance, research comparison, explicit caveats. | Documented here; paper results are contextual, not head-to-head. |

## 1.4 SCOPE AND BOUNDARIES

The implementation scope includes a browser interface, backend API, local image classification, automatic location-based public data retrieval, a nine-crop planner, experimental tabular yield regression, local agronomy passage retrieval, database migrations, and evaluation/report artifacts. It includes a starter knowledge library and a set of automated tests. It does not include an agronomist-approved nationwide recommendation product, a crop simulation model, a farmer accounting ledger, a current mandi-price predictor, a vector-search answer generator, or a deployment platform tested at national scale.

The scope boundary is important for judging the title “AI-powered.” The phrase describes the presence of machine-learning components in Modules 1 and the experimental part of Module 2. The simple crop planner is a rules-and-benchmark

system; Module 3 is deterministic retrieval. Not every result is produced by AI, and not every task is a prediction. The report uses more precise terms—classification, eligibility screening, ranking, yield regression and passage retrieval—where they fit.

| Included in current prototype | Not established by current evidence |
|---|---|
| Ten-class image screening for Tomato, Potato, Maize and Grape. | Field diagnosis across all crops, diseases, pests and nutrient problems. |
| Nine-crop rules-based shortlist using location and season inputs. | The best crop among all possible crops for every Indian region. |
| RF/XGBoost historical district yield estimates on supported candidates. | A validated farm-level yield, profit or crop-choice recommendation. |
| Curated English passages with source links, pages or sections. | Complete multilingual agronomy coverage or generative expert advice. |
| Automated code tests and saved metric artifacts. | A farmer trial, prospective agronomic validation or publication-level replication. |

## 1.5 USERS AND USE SCENARIOS

A farmer using the image screen may upload a clear leaf photograph, select the crop and receive the model's closest supported label. The uncertainty flag prompts a cautious interpretation, and the interface should be read as a screening aid. A farmer using the planner may search for a town or request current location, choose a sowing date, water availability and field area, and compare the crops that pass the planner's checks. A person using the advisor may type an agronomy question and inspect the matching stored passages and sources.

A student or evaluator may open the experimental Module 2 tabs to compare Random Forest and XGBoost rankings under identical farmer inputs. This is a testing view: its yield predictions come from historical district data and its benchmark surplus uses fixed reference prices and costs. The offline evaluation script is a separate tool for comparing algorithms on an identical holdout. The output of that script must not be assumed to be the live frontend model output, because the configurations and feature encoding differ.

## 1.6 REPORT ORGANIZATION

Chapter 2 reviews the technical ideas and research relevant to image classification, tabular crop planning, yield forecasting and retrieval. Chapter 3 describes requirements and the whole-system architecture. Chapters 4, 5 and 6 describe the three modules in implementation detail. Chapter 7 traces the browser, backend, database and external-service interactions. Chapter 8 documents the available

evaluation methods and results. Chapter 9 compares those results with published studies and discusses validity. Chapter 10 concludes and proposes next steps. The appendices provide evaluation formulas, test cases, data dictionaries, screenshot placeholders and reproducibility guidance.

# CHAPTER 2

# BACKGROUND AND LITERATURE REVIEW

## 2.1 PLANT DISEASE CLASSIFICATION FROM LEAF IMAGES

Image-based plant-disease screening commonly frames a leaf photograph as a supervised classification problem. A model receives pixel values and produces a score for each learned class. A training dataset must therefore define the supported crops and disease labels; a classifier cannot reliably report categories it was not trained to recognize. In this project, the class set is limited to ten labels across four crops. This choice makes a compact user workflow possible but excludes many crops, diseases and non-disease causes of leaf appearance.

PlantVillage established a widely used collection of healthy and diseased plant-leaf images. Mohanty, Hughes and Salathé (2016) report a CNN trained on 54,306 images covering 14 crop species and 26 diseases or healthy conditions, with 99.35% held-out accuracy across 38 classes. That result demonstrates performance on the paper's dataset split. It is not a guarantee of field accuracy. The distinction between image-level test performance and diagnosis under outdoor conditions matters because controlled backgrounds, lighting and single-leaf framing can simplify classification relative to field photographs. [3]

The supplied 2024 paper by Hukkeri et al. compares pretrained CNN architectures on a 38-class PlantVillage task and reports MobileNetV2 accuracy of 93.5%. Duhan et al. (2025) introduce an attention-enhanced lightweight model and report 99.87% for a MobileNetV2 baseline and 99.92% for their proposed RTR_Lite_MobileNetV2 in an augmented 39-class PlantVillage experiment. These numbers differ in dataset organization, class count, split, augmentation and training details. The report includes them as literature context and does not assert that the project's ten-class saved score is a controlled head-to-head improvement. [4] [5]

## 2.2 MOBILENETV2 AND TRANSFER LEARNING

MobileNetV2 is designed for efficient vision inference. Sandler et al. describe inverted residual blocks and linear bottlenecks: a narrow representation is expanded, processed with depthwise convolution and projected back to a narrow representation. These components reduce computation and memory use relative to conventional convolutional stacks. Transfer learning reuses an image representation learned on a large source task and adapts a classification head or selected layers to a smaller target dataset. This can reduce the need to train all parameters from random initialization, but performance still depends on the target dataset, preprocessing, class mapping, data split and fine-tuning decisions. [6]

For KrishiMitra, the MobileNetV2 architecture is the inference model associated with Module 1. The API applies image validity checks and transforms the accepted image into the representation expected by the model. The final output is a score vector over ten labels, converted to a top prediction and confidence value. The interface applies a separate uncertainty policy: it flags scores below 75% and crop mismatches. This policy is application logic layered on the classifier and is not part of the generic MobileNetV2 architecture.

Model confidence should not be confused with correctness probability unless confidence calibration has been measured. Softmax scores can be high for unfamiliar images because a closed-set model must distribute probability over known categories. An uncertainty flag is useful as a cautious user-interface signal but does not establish a formal reject option or calibrated clinical threshold. A stronger study would include reliability diagrams, expected calibration error, selective-risk curves and a genuine out-of-distribution test set.

## 2.3 CONTROLLED DATA AND FIELD DOMAIN SHIFT

A dataset shift occurs when images presented in deployment differ from training examples. Field images may include soil, sky, other leaves, shadows, glare, partial occlusion, multiple lesions, motion blur and varying camera distance. Disease appearance may vary with cultivar, growth stage and infection severity. If image backgrounds correlate with labels during training, a model can use background information rather than a disease-specific visual cue. This is one reason why

performance on a random held-out partition of a curated dataset should be supplemented by external field evaluation.

Singh et al. (2020) introduced PlantDoc as a visual plant-disease dataset assembled from web images, including 2,598 data points across 13 plant species and up to 17 disease classes. Its images are less uniform than controlled leaf datasets and illustrate the additional difficulty of in-the-wild detection. PlantDoc is a potential external benchmark only after reconciling label definitions and supported crops with the project's ten classes; using it without a label map would create an invalid comparison. [7]

A practical evaluation ladder for this project is: first, reproduce the existing within-dataset split; second, test duplicates and class overlap; third, measure out-of-domain performance on field photographs; fourth, review false negatives and uncertainty behavior with an agronomist. A high score at the first stage does not substitute for the later stages. Chapter 9 returns to this issue when comparing published CNN metrics.

## 2.4 CROP PLANNING AS A MULTI-CRITERIA DECISION

Crop recommendation may mean several distinct tasks. A classification dataset may label each soil-and-weather feature vector with one crop name. A rules engine may screen crops against agronomic constraints. A yield model may estimate expected production per area. A profit model may estimate returns after price and cost. A multi-criteria planner may rank options using yield, water, risk, rotation, prices and farmer preferences. These task definitions are not interchangeable. A yield forecast can inform a decision, but it does not on its own determine which crop should be planted or which option is profitable.

KrishiMitra's simple planner follows an explicit eligibility-and-ranking approach. It uses a limited catalogue of nine supported crops. It screens candidate crops using sowing-month windows, mapped pH, historical temperature and rainfall context, and water availability, then ranks eligible crops by benchmark surplus computed from national yield, price and cost data. This approach is interpretable: a user can inspect which checks were passed or raised as cautions. Its limitation is that a broad set of ranges and national benchmarks cannot represent every district, variety, soil profile, local price or farm budget.

The planner should therefore be understood as a provisional comparison workflow, not a comprehensive crop optimizer. A shortlist may contain fewer than three options, and missing required data may prevent a ranking. A public map estimate is not a laboratory soil analysis. Forecast weather is not the same as historical climate, and a seven-day rain forecast cannot be compared to annual crop rainfall requirements. The code uses separate time scopes to avoid this unit mismatch.

## 2.5 RANDOM FOREST FOR TABULAR REGRESSION

Random Forest is an ensemble of decision trees trained with randomized data and feature selection. For regression, individual tree estimates are aggregated, commonly by averaging. Bagging and random feature selection can reduce variance compared with a single deep tree. Random Forest can model nonlinear relationships and interactions without standardizing numerical features. It is often a useful baseline for mixed tabular data, but it can produce poor extrapolations when future conditions differ from the training distribution, and it does not automatically solve missing, biased or weakly informative data. [8]

The project's offline RF model receives state, district, crop and year. Since state, district and crop are categorical, the evaluation pipeline encodes those fields. Year is represented categorically in the offline comparison, while the live experimental service uses year numerically. The production-tab model and the offline score are thus not the same exact configuration. The main purpose of the offline comparison is to put Random Forest and XGBoost on a common test set, not to certify the frontend's ranking accuracy.

## 2.6 XGBOOST FOR TABULAR REGRESSION

XGBoost is a scalable gradient-boosted tree system. Boosting builds trees sequentially so that later trees improve residual errors from earlier trees, while regularization and tree constraints help control model complexity. It supports efficient training on tabular features and can capture nonlinear patterns. It remains sensitive to dataset quality, target definition, feature leakage, hyperparameters and validation design. A newer algorithm is not automatically superior to a well-chosen baseline on a particular dataset. [9]

In the recorded experiment, XGBoost used 120 estimators in the offline run and 60 estimators in the UI model. Its depth and learning-rate settings were fixed, with

subsampling/column-sampling settings documented in the project code. Random Forest used 120 trees offline and 60 trees in the UI. The recorded offline run showed shorter training and prediction time for XGBoost on that machine, while Random Forest had slightly lower MAE and RMSE. These are experiment-specific outcomes, not universal properties of either algorithm. [10]

## 2.7 HISTORICAL YIELD FORECASTING RESEARCH

Crop-yield forecasting papers differ in region, crop, observation unit, prediction horizon, features and target scaling. Jhajharia et al. (2023) evaluate methods for five crops in Rajasthan and report that Random Forest performed best among their compared methods, with $R^2$ 0.963, RMSE 0.035 and MAE 0.0251. The available abstract does not make those metric units directly equivalent to the project's kg/ha absolute errors. Direct numerical ranking would be unsound without matching target transformation and test protocol. [11]

De Clercq and Mahdi (2024) study district-level rice yield prediction for India using climate reanalysis and earth-observation data across 247 rice-producing districts and 20 years. Their reported best out-of-sample values reach $R^2$ 0.82, MAE 0.29 and MAPE 0.16. The study uses temporally meaningful forecast features, a rice-specific target and a different model comparison. It is a useful methodological reference for future KrishiMitra work, but it is not a direct benchmark for a general multi-crop yield table with state/district/crop/year categorical predictors. [12]

A defensible comparison requires the same prediction task and a consistent test design. If results come from different papers, the report should describe the task and protocol alongside the metric. If a score is normalized, converted or calculated on a different scale, that detail must be stated. An evaluation matrix can clarify the literature; it cannot make unlike data comparable by itself.

## 2.8 TEXT RETRIEVAL, RAG AND CITATION-GROUNDED ANSWERS

Full-text retrieval identifies text passages that match query terms. PostgreSQL represents indexed documents as tsvector values and queries as tsquery values, then evaluates matches and ranks candidates. Explicit synonyms can broaden a user query by mapping selected common terms to a documented vocabulary. This approach is fast, local and auditable, but it does not understand arbitrary paraphrases in the way a

semantic model might. A match score is a rank signal rather than a confidence probability.

RAG, as described by Lewis et al. (2020), combines retrieval with a generator that conditions its output on retrieved passages. Earlier KrishiMitra work considered or used a Gemini-based text-answer direction and discussed RAG as a grounding approach. The current advisor has changed to a retrieve-and-return design. It sends the matched stored passage to the user with source metadata and does not synthesize an answer. RAG remains part of the history/background because it explains the evolution of the feature, but it is not emphasized as a current capability. [13]

This distinction changes how evaluation should be designed. For retrieval-and-return, the main questions are whether the correct passage is found, whether its citation is accurate, whether the system abstains when evidence is missing, and how quickly the result appears. For a generative RAG system, answer faithfulness and unsupported-claim rates would also be required. The current system has not received the first category's benchmark because its configured database schema did not satisfy the active query path during the saved attempt.

## 2.9 RESEARCH GAP AND PROJECT POSITION

The project's academic contribution is not a claim that a new model beats the state of the art. Its present value is the integration and transparent comparison of three distinct information workflows, together with explicit limitations and a common Random Forest versus XGBoost temporal holdout. The evaluation exposes an important result: both models improve on the training-mean baseline in absolute error but achieve low $R^2$ on the available data. This suggests that the dataset and predictors do not explain most variation in held-out yields under this setup.

The gap between local and published scores also suggests what must improve before claiming better performance: define one task, obtain matched features and targets, fix a reproducible split, tune within training data, compare to simple and published baselines, report uncertainty, and test external or future groups. The evaluation matrix in Chapters 8–9 is therefore designed to support honest comparison rather than select papers with favorable scores or weak implementations.

CHAPTER 3

# REQUIREMENTS AND SYSTEM ARCHITECTURE

## 3.1 SYSTEM CONTEXT AND OPERATING ENVIRONMENT

The application is divided into a browser interface, a FastAPI backend, module-specific Python services, PostgreSQL persistence and external public data sources. The browser is responsible for presenting forms, selecting user actions, rendering result cards and giving feedback on pending or failed requests. It does not calculate agronomic recommendations or run the crop disease model. The backend owns validation and domain logic. PostgreSQL stores selected durable records and advisor content. Public services are called only when the relevant planning request is initiated.

The development workflow uses Docker Compose to run PostgreSQL with a pgvector-capable image. The frontend and backend are started as separate processes through project scripts. Alembic applies database schema migrations when the application starts. This architecture keeps the test database and application services separable, but it means a stale local database can cause a feature to fail even when the code path exists. The Module 3 benchmark failure is a concrete example of the importance of schema-version checks in the environment.

*Figure 3.1. Overall KrishiMitra system architecture and separation of module services.*

## 3.2 FUNCTIONAL REQUIREMENTS

| ID | Requirement | Verification evidence |
|---|---|---|
| FR-01 | Accept a supported image only after format, size and decoding validation. | Module 1 validation tests and API behavior. |
| FR-02 | Run supported leaf images through the ten-label MobileNetV2 classifier. | Saved model metrics; application response. |
| FR-03 | Mark low-confidence or crop-conflict output uncertain. | Unit/API tests and UI state. |
| FR-04 | Find a location from a selected town or explicit GPS request. | Geocoder/GPS integration behavior. |
| FR-05 | Fetch public soil and weather data for a planning request. | Planning smoke checks and failure handling. |
| FR-06 | Screen supported crops and rank eligible candidates by benchmark surplus. | Planner unit tests and displayed explanations. |
| FR-07 | Expose RF and XGBoost experimental results for comparison. | Frontend tabs and yield-model service. |

| | | |
|---|---|---|
| FR-08 | Retrieve advisor passages and citations or abstain. | Code path; full DB-backed benchmark remains pending. |
| FR-09 | Persist applicable scan/advisor metadata. | Database schema and migration artifacts. |
| FR-10 | Do not store the uploaded leaf image with scan metadata. | Storage design and route implementation review. |

## 3.3 NON-FUNCTIONAL REQUIREMENTS

| Quality attribute | Design requirement | Current evidence / limitation |
|---|---|---|
| Usability | Clear module navigation, readable result cards and form validation. | Frontend tests and responsive checks recorded in Module 2 review. |
| Reliability | Missing public values should be shown as missing, not replaced by guesses. | Planner rejects incomplete data and has retry behavior for selected transient failures. |
| Privacy | Location access occurs only after a user action; uploaded leaf images are not stored with scans. | Documented implementation behavior; deployment privacy review remains necessary. |
| Traceability | Advisor passages include sources and page/section when available. | Stored chunk/document metadata; small corpus. |
| Maintainability | Separate Python modules and schema migrations. | Module boundaries and Alembic startup workflow. |
| Performance | Responsive local model and API behavior. | Saved latency for MobileNet; environment-specific and not a service-level guarantee. |
| Portability | Windows start scripts and containerized database. | Frontend/backend are not fully containerized as a single deployment. |

## 3.4 DATA SOURCES AND SERVICE CONTRACTS

| Source | Fields used | Interpretation and limitation |
|---|---|---|
| OpenLandMap | pH, organic carbon, sand at 10 cm. | Nearby nominal 250 m estimated map cell, not current test; only pH used for eligibility. |
| Open-Meteo forecast | Current conditions and seven-day forecast. | Short-horizon weather; forecast cautions do not replace crop climate ranges. |
| Open-Meteo historical / ERA5 | Daily historical temperature and precipitation aggregated by month/year. | Reanalysis climate; typical history is not a future-season guarantee. |
| FAO EcoCrop | Indicative crop pH, temperature and annual rainfall ranges. | Crop catalogue context; broad ranges do not model variety/stage/district specifics. |
| Government ag statistics | National multi-year yield references. | National averages, not specific to the requested field. |
| Government MSP / A2+FL cost | Reference price and production-cost benchmarks. | Static dated values; not live mandi price or complete farmer budget. |
| TCI Cornell yield CSV | State/district/year/crop/yield table. | Historical district observations used in experimental |

| | | |
|---|---|---|
| | | regression; data ends 2017. |

## 3.5 DATA FLOW AND TRUST BOUNDARIES

A request enters the system from the browser and crosses a trust boundary at the API. Input validation rejects unsupported or malformed payloads before module code processes them. Where the user requests public information, the backend sends coordinates to named services. The service response is validated for missing values, ranges and expected time keys before the planner uses it. A successful public response is evidence about the service result, not proof that the mapped environmental estimate matches the field.

The crop-health image passes through another boundary. The backend validates the bytes and image dimensions, performs suitability checks and calls the classifier. The uploaded image is held for inference and is not persisted with the scan record. The database receives the metadata required for history and traceability, including an image hash. The hash is a stable identifier of the file contents; it is not an anonymized image and may reveal equality across uploads if exposed. The project should protect metadata access and define retention policy in deployment.

The advisor database contains curated source metadata and passage text. A retrieved result is an excerpt or paraphrase already stored in that corpus. The current text route does not generate an original agronomic response. Citations map returned content to the stored source. Incorrect seed metadata, old source material, or a missing passage can still produce incomplete advice; the interface should show the source and encourage local verification.

## 3.6 FAILURE HANDLING AND DEGRADED MODES

External dependencies can be unavailable, slow or incomplete. The planner uses bounded retries for selected transient network responses and validates expected weather periods and soil fields. When required data is missing, it returns an insufficient-data state rather than silently setting a value to zero. A historical-only provisional shortlist may be shown in the clearly marked supported path when forecast data is unavailable; it should not be mistaken for a current forecast-based result. Incomplete soil results are not cached as complete data.

The database is also a dependency. If the schema does not match the expected SQL model, the advisor's retrieval request can fail before relevance is measured. The correct operational response is to apply migrations and verify schema state, then seed the curated library. Reporting a retrieval accuracy score in that condition would be impossible. Other failure conditions include malformed JSON, user cancellation, stale frontend responses, geocoder ambiguity and unsupported crop names. The application contains safeguards for several of these cases; future deployment tests should exercise them end-to-end.

## 3.7 SECURITY, PRIVACY AND RESPONSIBLE USE

The application accepts user-provided images and coordinates and uses a database to persist selected records. The design avoids storing the photo with scan metadata and requests location only after explicit action. These are useful minimization steps but do not replace standard web security practices. A deployed system must use authenticated database credentials, restrict administrative access, protect API logs, use HTTPS, configure CORS deliberately, rotate secrets and avoid logging image bytes or unnecessary location precision.

Agronomy results are decision support, not regulated diagnosis or individualized extension advice. The uncertainty flag and source citations help users understand the output, but the system should not prescribe pesticide or fertilizer quantities without reviewed, region-specific, legally current sources. The advisor intentionally blocks dosage requests. Disease results should be confirmed by a trained person when action is consequential. Economic benchmarks should be clearly marked as scenarios so that no user mistakes them for guaranteed profit.

**PASTE AUTHENTIC SCREENSHOT HERE**

**Figure 3.2. Frontend home and navigation**

**Capture the running application and module navigation.**

**CHAPTER 4**

# MODULE 1: CROP HEALTH SCREENING

## 4.1 PURPOSE AND USER WORKFLOW

Module 1 accepts a leaf photograph for one of four selected crops: Tomato, Potato, Maize or Grape. The supported classifier labels describe healthy leaves and selected disease categories. The user selects the crop and submits a clear image. The backend checks the uploaded file before invoking MobileNetV2. If the image passes the checks, the model returns scores over the ten labels. The service selects the class with the highest score and compares the predicted crop with the user-selected crop.

The result is intentionally framed as a screening aid. An image classifier can help a farmer notice a possible symptom category, but visual similarity does not establish a pathogen diagnosis. A field image may contain several symptoms or causal factors. The top prediction can be wrong even at high confidence, particularly for a supported class absent from the real cause. Users should interpret low confidence and crop conflict as uncertainty and should seek confirmation for consequential plant-protection decisions.

| Label family | Classes |
|---|---|
| Tomato | Tomato healthy; Tomato Early Blight; Tomato Late Blight |
| Potato | Potato healthy; Potato Early Blight; Potato Late Blight |
| Maize | Maize healthy; Maize Common Rust |
| Grape | Grape healthy; Grape Black Rot |

## 4.2 MODEL BLOCK DIAGRAM AND INFERENCE STAGES

*Figure 4.1. Full Module 1 image screening workflow with separate uncertainty and persistence outputs.*

## 4.3 INPUT VALIDATION

The first stage prevents malformed files from reaching model inference. The backend requires an acceptable image content type, checks file size and attempts to decode image bytes. A corrupted file or unsupported format is rejected. Additional image suitability checks screen out obvious non-leaf or unsuitable inputs using basic visual cues. These checks are not a trained crop detector and are not equivalent to verifying that a specialist would consider the image diagnosable.

Validation should be evaluated independently from classification. A full module assessment should count invalid uploads correctly rejected, valid supported leaves

accepted, unsupported objects incorrectly accepted, and field images rejected even though they are valid. False rejection can frustrate a farmer; false acceptance can produce a misleading label. The current stored MobileNet metric table does not report this validation-stage performance, so no acceptance or rejection accuracy is claimed.

## 4.4 PREPROCESSING AND MODEL INFERENCE

Accepted images are transformed into the input representation used by the MobileNetV2 checkpoint. Inference produces a vector of class scores. The largest score identifies the closest class in the supported label list. The API returns the label and confidence to the frontend, where the result is displayed with uncertainty state and explanatory text. The model is an image-classification system, not an object detector: it does not localize a lesion with bounding boxes or segment symptomatic pixels.

A reliable reproduction requires matching the model file, class order, image decoder, preprocessing pipeline, output activation and evaluation labels. If class order changes while the output layer remains the same size, the classifier may appear to run while assigning incorrect names to its output indices. This is why the class map and checkpoint must be archived together. A complete model card should record framework versions, checkpoint checksum, image-size policy, color-channel order, normalization, trained label order and hardware/runtime used for latency measurement.

## 4.5 UNCERTAINTY POLICY

The service sets an uncertain state when the top score is below 0.75 or when the crop selected by the farmer conflicts with the crop inferred from the model's top label. This combines two distinct signals. The score condition asks whether the model is sufficiently confident according to a chosen threshold. The crop condition asks whether the user's context and the predicted class agree. Neither signal can prove that the label is right. The UI should maintain the distinction between the highest-scoring class and the uncertainty message.

A fixed threshold is easy to explain but may behave differently across classes. If the model is overconfident, a 75% threshold can still admit wrong predictions. If scores are poorly calibrated, a class-specific threshold could be more appropriate, but that would require a calibration set that is separate from the final test set. A proper reject-

option study would plot coverage against error among accepted predictions, report uncertainty separately for each class, and evaluate the crop-mismatch branch with a designed set of conflicting selections.

## 4.6 PERSISTENCE AND IMAGE RETENTION

The scan record stores prediction-related metadata and an image hash. The uploaded photo is not retained with that metadata. This reduces the amount of image content stored in the database and reflects a data-minimization design. The hash can be used to identify identical file bytes in the metadata stream, but it is still a persistent identifier. Applications should limit access to scan history and should define the record's retention period. System logs should not accidentally persist the multipart image body or filesystem temporary copies.

The distinction between transient processing and persisted records should be described consistently in user-facing privacy text. The project's statement is narrow: it does not save the uploaded leaf photo as part of the scan metadata. It should not be broadened into a claim that no component ever buffers image bytes, that server logs cannot contain a request, or that all data are automatically deleted. Deployment configuration must be reviewed separately.

## 4.7 DATA SPLIT AND SAVED EVALUATION

The saved training report identifies a PlantVillage-derived ten-class test split with 7,427 training images, 1,591 validation images and 1,591 test images. The test set is the source of the saved classification metrics. During this report update, the corresponding test image files were missing under the processed data directory, so the metrics could not be independently recomputed. The project folder contains the evaluation summary and per-class metrics, but the image evidence needed for a rerun is incomplete.

| Metric | Saved value | Interpretation |
|---|---|---|
| Test samples | 1,591 | Saved held-out image count. |
| Accuracy | 98.6801% | Overall correct classification fraction. |
| Macro precision | 98.9150% | Unweighted mean precision across classes. |
| Macro recall | 98.3695% | Unweighted mean recall across classes. |
| Macro F1 | 98.6264% | Unweighted mean F1 across classes. |
| Median latency | 40.85 ms | Saved p50 inference latency. |

| | | |
|---|---|---|
| 95th percentile latency | 43.52 ms | Saved p95 inference latency. |

*Table 4.2. Saved MobileNetV2 evaluation report. These values were not recomputed for the revised report.*

## 4.8 PER-CLASS METRICS AND SUPPORT

Macro averages are helpful when classes have different sample counts, but they can conceal weaker performance on a particular class. The saved per-class table reports support, precision, recall and F1. Support is the number of ground-truth examples. Recall is particularly relevant for screening because a low value means a portion of examples from that class were missed. Small support makes a class-level estimate less stable. The potato-healthy class has only 23 test cases, so its metric should not be read as a highly precise population estimate.

| Class | Precision | Recall | F1 | Support |
|---|---|---|---|---|
| Tomato healthy | 0.9917 | 1.0000 | 0.9958 | 239 |
| Tomato Early Blight | 0.9858 | 0.9267 | 0.9553 | 150 |
| Tomato Late Blight | 0.9724 | 0.9860 | 0.9792 | 286 |
| Potato healthy | 1.0000 | 0.9565 | 0.9778 | 23 |
| Potato Early Blight | 0.9804 | 1.0000 | 0.9901 | 150 |
| Potato Late Blight | 0.9669 | 0.9733 | 0.9701 | 150 |
| Maize healthy | 0.9943 | 1.0000 | 0.9971 | 174 |
| Maize Common Rust | 1.0000 | 0.9944 | 0.9972 | 179 |
| Grape healthy | 1.0000 | 1.0000 | 1.0000 | 63 |
| Grape Black Rot | 1.0000 | 1.0000 | 1.0000 | 177 |

*Table 4.3. Stored classwise scores. The class names are shown in human-readable form.*

Figure 4.2. Saved per-class MobileNetV2 F1 scores and test support.

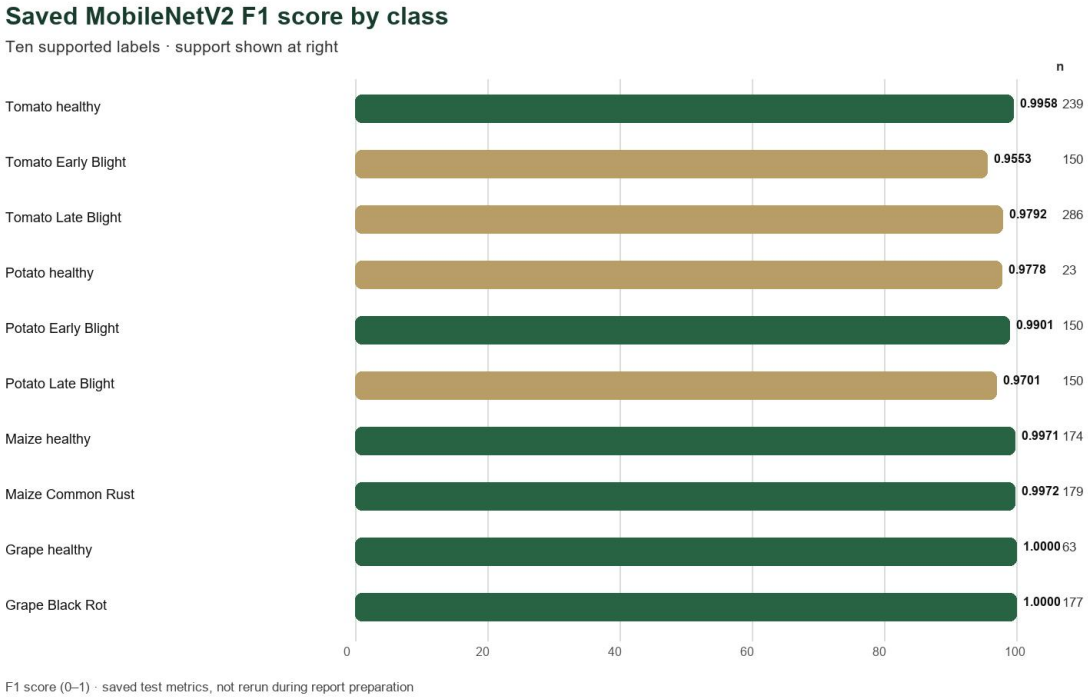



Table 4.4. MobileNetV2 result compared with selected plant-disease studies.

| Study | Dataset / task | Reported result | Comparison note |
|---|---|---|---|
| KrishiMitra (saved) | PlantVillage-derived; 10 labels; 1,591 test images | Accuracy 98.68%; macro F1 98.63% | Saved metrics; test images unavailable for rerun. |
| Mohanty et al. (2016) [3] | PlantVillage; 38 classes | Accuracy 99.35% | Different labels and split. |
| Hukkeri et al. (2024) [4] | PlantVillage; MobileNetV2; 38 classes | Accuracy 93.5% | Different protocol and classes. |
| Duhan et al. (2025) [5] | Augmented 39-class task; MobileNetV2 baseline | 99.87% baseline; 99.92% proposed | Different augmentation and split. |

Published results provide context, not a leaderboard: class count, image partitions and augmentation affect task difficulty. A superiority claim requires a shared test set and a matched evaluation protocol.

Figure 4.3. Stored MobileNetV2 accuracy beside selected literature scores; protocols differ.

| | |
|---|---|
| PASTE AUTHENTIC SCREENSHOT HERE<br>Figure 4.4. Module 1 prediction result<br>Show supported crop, label and confidence. | PASTE AUTHENTIC SCREENSHOT HERE<br>Figure 4.5. Module 1 uncertainty result<br>Show genuine low-confidence or crop-conflict example. |

## 4.9 LIMITATIONS AND IMPROVEMENT PLAN

- Restore the exact train/validation/test image folders and archive their checksums with the split manifest; reproduce all metrics from the same checkpoint.
- Add an external field-image test set with farmer consent and specialist-verified labels. Maintain a label map that matches the ten supported classes and define how ambiguous or mixed cases are handled.
- Report a confusion matrix, classwise confidence intervals, calibration metrics, macro and weighted scores, and selective risk when the system abstains.
- Evaluate image-quality rejection separately from disease classification. Include blur, low light, multiple leaves, non-leaf objects and supported crops photographed in varied conditions.
- Review disease advice and result text with plant pathologists and local KVK specialists. Keep the classifier output separate from treatment recommendations.

# CHAPTER 5

# MODULE 2: SOIL AND CROP PLANNING

## 5.1 USER INPUTS AND PLANNING CONTEXT

The planning screen requests a location, planned sowing date, water availability and field area. Location can be selected from a search result or through an explicit current-location action. Area is provided in the supported unit and converted to hectares for internal calculations. The farmer selects whether irrigation is available, whether the plan is rainfed, or whether water availability is not yet known. These choices affect screening and cautions; they are not measured water volumes or a forecast of irrigation reliability.

| Input | Representation / unit | Use in planner | Boundary |
|---|---|---|---|
| Location | Selected town/village coordinates. | Requests mapped soil and weather. | Nearby raster cell, not survey-grade field boundary. |
| Sowing date | Calendar date. | Selects crop window and weather context. | Broad indicative windows, not a district variety calendar. |
| Water availability | Unknown / rainfed / irrigation available. | Controls dry-month screening and cautions. | No irrigation quantity, pump capacity or stored soil water. |

| | | | |
|---|---|---|---|
| Field area | Area and selected unit. | Scales benchmark surplus to a farm area. | Does not estimate actual plot yield or all production costs. |

## 5.2 SOIL DATA SOURCE AND LIMITS

OpenLandMap provides map estimates for pH, organic carbon and sand at 10 cm depth, associated with a nearby nominal 250-metre cell. The raster product is a spatial estimate informed by historical observations, not a fresh sample taken from the farmer's plot. A village-level location may cover different soil conditions across a small area, and the selected coordinate may not coincide with the actual cultivated parcel. The interface should identify soil values as mapped estimates and not as a substitute for a laboratory soil test or Soil Health Card. [14]

Only pH currently participates in the crop-screening checks. Organic carbon and sand are presented as context; the application does not infer nitrogen, phosphorus, potassium, micronutrients or electrical conductivity from those values. This matters because plant-available nutrients depend on measurements and methods not represented by the selected map properties. A future integration should retain units, depth, dates, source and spatial resolution with every feature and validate the model estimate against permitted field samples.

## 5.3 WEATHER AND CLIMATE SOURCES

Open-Meteo provides current conditions and a seven-day forecast for the requested coordinates. Historical weather is used for typical climate summaries, including temperature and rainfall over ten complete past calendar years from ERA5. A forecast describes the near term; a climate summary describes a historical distribution or average. Neither is a deterministic prediction of the full crop season. When a future sowing date falls beyond the forecast window, the planner relies on historical climate for its longer-horizon checks and presents forecast conditions only where date ranges overlap. [15] [16]

The service validates dates, expected daily entries, units, low/high temperature ordering and non-null rainfall. A missing rainfall value is not treated as zero. This avoids a dangerous failure mode in which unavailable observations appear as drought. Historical climate provides context but can hide year-to-year extremes. The current

planner does not estimate forecast uncertainty or simulate future climate under different scenarios.

## 5.4 CROP CATALOGUE AND AGRONOMIC CHECKS

The catalogue supports wheat, chickpea (gram), rapeseed/mustard, maize, sorghum, pearl millet, groundnut, soybean and sunflower. Crop ranges are drawn from FAO EcoCrop and source-checked project data. The planner first checks broad sowing-month windows. It then checks mapped pH against an absolute crop range and evaluates monthly historical temperature during the first four growth months. The four-month mean is compared with a published optimal temperature range, and each monthly mean is checked against absolute limits. This is a transparent screen, not a detailed crop-growth model. [17]

Rainfall screening compares annual rainfall with the catalogue's annual range rather than comparing seven-day forecast rain to annual needs. The planner additionally uses a fixed 30 mm monthly threshold as a dry-month heuristic over the first four months. Under rainfed selection, low annual or monthly rain can exclude a crop. Irrigation availability or unknown water status can preserve a candidate but add an irrigation condition. Excess annual rain can add a drainage caution. The threshold is an application heuristic and must not be cited as a universal crop water requirement.

| Screen | Data used | Decision role | Interpretive limit |
|---|---|---|---|
| Sowing window | Selected date month and crop catalogue. | Exclude clearly off-window crops. | Broad season assumptions, not district calendar. |
| Mapped soil pH | OpenLandMap pH estimate, converted to pH scale. | Compare against crop absolute limits. | One map cell and one depth; not field test. |
| Temperature | Historical monthly means; four-month growth window. | Compare monthly absolute and four-month optimal ranges. | Simplified climate proxy; not phenology simulation. |
| Annual rainfall | Historical annual totals. | Compare crop annual rainfall interval. | Does not represent soil water, runoff or irrigation. |
| Dry-month heuristic | Historical growing-month rainfall <30 mm. | Flag/exclude under rainfed conditions. | Project heuristic, not official universal threshold. |
| Forecast | Current and seven-day weather. | Add near-term sowing caution. | Not whole-season forecast. |

## 5.5 SIMPLE ENGINE RANKING AND BENCHMARK ECONOMICS

Eligible crops are ranked using a benchmark-surplus formula derived from published national yield, a reference price and an A2+FL production-cost benchmark. The yield per hectare is converted from kilograms to quintals by dividing by 100. The difference between reference price and cost per quintal is then multiplied by yield in quintals per hectare. The planner can scale the per-hectare benchmark to the requested field area. It presents up to three supported candidates and allows fewer options when only a smaller number pass the checks.

For wheat, the catalogue value is 3,547 kg/ha, which equals 35.47 quintals/ha. With a reference price of ₹2,610/quintal and cost of ₹1,264/quintal, benchmark surplus is 35.47 × (2,610 − 1,264) = approximately ₹47,743 per hectare. This value is before full land and capital opportunity costs and is not an individual farm budget. It excludes actual local yield, sale price, input rates, labor, transport, harvesting losses, storage, credit costs and market access. A stress scenario is a sensitivity example, not a calibrated probability interval. [18] [19]

| Economic element | Catalogue basis | What is represented | What is not represented |
|---|---|---|---|
| Yield | Government national multi-year yield reference. | Indicative benchmark crop productivity. | Farmer-specific field yield or weather-adjusted forecast. |
| Price | Published dated MSP/reference value. | A policy/benchmark price scenario. | Live mandi, negotiated or guaranteed sale price. |
| Cost | Published A2+FL cost per quintal. | A national cost benchmark. | Detailed cost of this farm, full C2 or opportunity cost. |
| Area | Farmer-entered area converted to hectares. | Linear scaling of benchmark surplus. | Yield response to plot size or economies of scale. |
| Ranking | Eligibility then descending benchmark surplus. | Transparent comparison among supported crops. | Expected profit probability or optimized multi-year rotation. |

## 5.6 FAILURE STATES AND SHORTLIST INTERPRETATION

The planner may return no ranking if a required soil or climate input is unavailable. This avoids using guessed values to create a false sense of precision. A temporary forecast outage can support a historical-only provisional shortlist if that state is explicitly shown; it does not become a current weather forecast. A complete soil result

is needed where the pH screen requires it. The user should be able to distinguish an excluded crop, a passing crop with caution, and a candidate whose result is provisional because an input source is missing.

A shortlist is limited to nine supported crops and to current catalogue assumptions. It should not be described as "the best three crops in India" or a guarantee of suitability. Conditions such as rotation history, seed availability, pest pressure, irrigation costs, labor, market access and farmer risk tolerance are not comprehensively represented. Local extension services remain necessary for decisions with significant financial or agronomic impact.

## 5.7 EXPERIMENTAL YIELD MODELS AND DATASET

The experimental Module 2 path reads a local district-level historical yield table, distributed through TCI Cornell's data collection. The features in the offline comparison are state, district, year and crop; the target is yield in kilograms per hectare. The model service retains supported crop variants and removes invalid or nonpositive yield records. The live UI training process limits examples per crop to control training size. The dataset is historical and its records end in 2017. It does not contain the farmer's measured soil nutrients, current weather, field irrigation, sowing date or actual costs. [20]

The common offline evaluation uses a chronological holdout: data from 1966 through 2012 train the models and 2013 through 2017 form the test period. There are 89,651 training rows and 10,011 test rows, with 304 districts represented in both. The held-out years are later than the training period, which is preferable to a random row split for a forecasting-like problem. However, district overlap means the experiment does not measure performance in entirely new districts. Hyperparameters were fixed rather than tuned in a nested temporal procedure.

The data table includes realized yields and geographic/crop labels. Because year and district identifiers strongly structure historical averages, a model can learn location-specific baseline patterns without learning how soil, rainfall or management cause yield changes. This may explain why the temporal holdout errors are still large and $R^2$ remains low. The regression target is crop yield, not a label for "most profitable crop." To turn yield into a model-based ranking, the UI multiplies predicted yield by

the same published price-minus-cost benchmark used in the simple engine. That downstream score adds no local price or cost evidence.

| Model | Offline configuration | In-application configuration | Notes |
|---|---|---|---|
| Random Forest | 120 trees; categorical state/district/year/crop encoding. | 60 trees; up to 2,500 samples per crop; year numeric. | Different encoding and sample cap; do not equate metrics. |
| XGBoost | 120 estimators; depth and learning settings fixed. | 60 estimators; up to 2,500 samples per crop; year numeric. | CPU-oriented experimental service. |
| Mean baseline | Training-target mean. | Not the user-facing planner. | Reference point for offline error comparison. |

Table 5.1. Offline yield-regression results on the shared chronological test set.

| Model | MAE kg/ha ↓ | RMSE kg/ha ↓ | $R^2$ ↑ | Fit sec | Predict sec |
|---|---|---|---|---|---|
| Mean baseline | 770.31 | 1,252.26 | −0.2075 | — | — |
| Random Forest | 676.89 | 1,071.50 | 0.1159 | 35.037 | 0.061 |
| XGBoost | 683.10 | 1,080.97 | 0.1003 | 0.404 | 0.020 |

Offline comparison uses 89,651 training rows from 1966–2012 and the same 10,011 held-out rows from 2013–2017 for both models. Random Forest has slightly lower errors on this run; this does not establish general superiority. UI model settings are separate.

Table 5.2. Separate live UI examples for three engines (1.00 ha scenario).

| Engine | Rank | Crop | Predicted yield kg/ha | Benchmark surplus ₹/ha |
|---|---|---|---|---|
| Simple planner | 1 | Wheat | 3,547 catalogue yield | 47,743 |
| Simple planner | 2 | Rapeseed / mustard | Catalogue yield | 47,651 |
| Simple planner | 3 | Maize | Catalogue yield | 45,084 |
| Random Forest UI | 1 | Wheat | 3,486.4 | 46,927 |
| Random Forest UI | 2 | Chickpea (gram) | 1,138.7 | 25,131 |
| Random Forest UI | 3 | Rapeseed / mustard | 764.9 | 24,828 |
| XGBoost UI | 1 | Rapeseed / mustard | 1,061.7 | 34,461 |
| XGBoost UI | 2 | Sunflower | 1,096.3 | 30,488 |
| XGBoost UI | 3 | Wheat | 2,089.5 | 28,124 |

These are distinct interface examples, not accuracy scores or a matched field trial. The simple engine uses catalogue yields; the ML engines use predicted yields and apply fixed published price-minus-cost benchmarks. Kept separate from the offline test results above.

Figure 5.1. Separate simple, Random Forest UI and XGBoost UI benchmark-surplus examples.

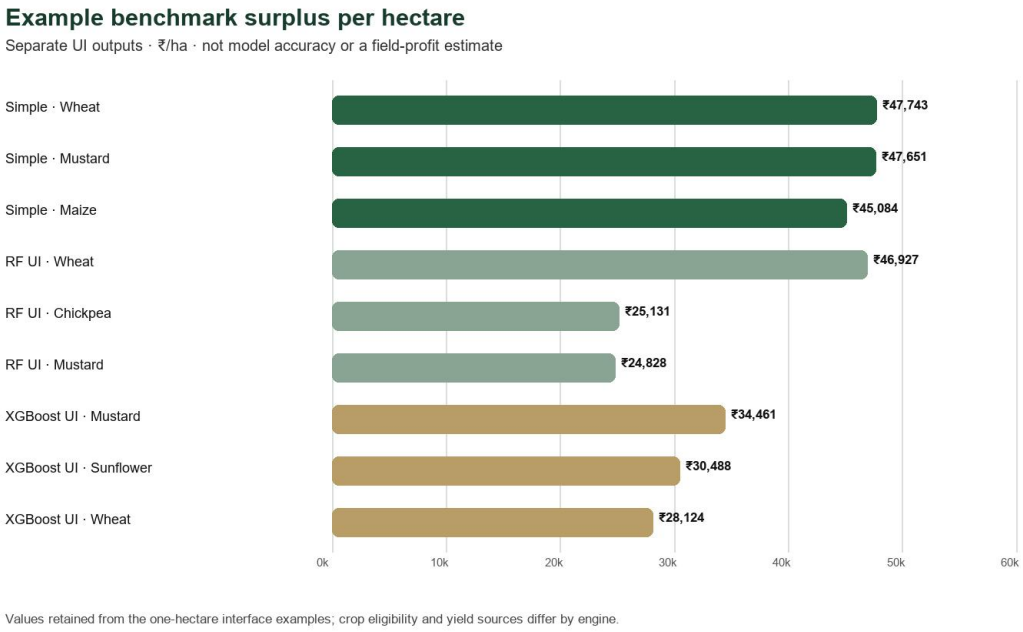



Table 5.3. Selected yield and crop-recommendation studies compared with KrishiMitra.

| Study | Task / scope | Reported result | Comparison note |
|---|---|---|---|
| KrishiMitra RF | District crop-yield regression; 1966–2017 | MAE 676.89 kg/ha; RMSE 1,071.50; $R^2$ .1159 | Internal paired holdout. |
| KrishiMitra XGBoost | Same target, features and test rows | MAE 683.10 kg/ha; RMSE 1,080.97; $R^2$ .1003 | Internal paired holdout. |
| Jhajharia et al. (2023) [11] | Five Rajasthan crops; RF yield study | $R^2$ .963; RMSE .035; MAE .0251 (abstract scale unclear) | Different dataset/units; not comparable numerically. |
| De Clercq & Mahdi (2024) [12] | Indian district rice yield; climate + remote sensing | Out-of-sample $R^2$ .82; MAE .29; MAPE .16 | Different target, features and metric units. |
| Thihlum et al. (2025) [21] | Mizoram crop-label classification; RF/PSO + XGBoost | Test accuracy .98 | Classification score, not yield-regression error. |

The studies solve different tasks. Crop-label accuracy, scaled errors from other yield datasets and this project's errors do not share a denominator; this report does not claim to outperform them.

Figure 5.2. Offline Module 2 error and $R^2$ metrics on the shared temporal test set.

| | |
|---|---|
| PASTE AUTHENTIC SCREENSHOT HERE<br>Figure 5.3. Simple planner output<br>Show inputs, shortlist and benchmark disclaimer. | PASTE AUTHENTIC SCREENSHOT HERE<br>Figure 5.4. Random Forest UI result<br>Use same inputs as XGBoost; mark experimental. |
| PASTE AUTHENTIC SCREENSHOT HERE<br>Figure 5.5. XGBoost UI result<br>Match location, season, area and water inputs. | PASTE AUTHENTIC SCREENSHOT HERE<br>Figure 5.6. Offline evaluation matrix<br>Show metrics and chronological split. |

## 5.8 LIMITATIONS AND SAFE INTERPRETATION

A high predicted historical yield is not evidence that a crop will be profitable on a particular field. Yield is affected by features missing from the model, including soil nutrients, variety, planting density, irrigation, pests and weather at critical growth stages. The dataset may have measurement inconsistencies or changing district boundaries over decades. The temporal split still contains known districts, and there is no spatially held-out test. The target values may exhibit heavy tails and crop-specific variance, so overall metrics can hide severe errors on less common crops.

The model comparison is useful as a controlled internal test of two algorithms on one common held-out dataset. It does not prove that the entire planner is better than a published crop recommender. The simple engine ranks only candidates that pass rules; the ML models estimate yield for historical crop records. These tracks have different goals and need separate metrics. A field study should compare actual crop choices and outcomes under a defined protocol before making adoption or profitability claims.

# CHAPTER 6

# MODULE 3: LOCAL AGRONOMY ADVISOR

## 6.1 FEATURE PURPOSE AND RESPONSE CONTRACT

The advisor is intended to help a user locate relevant agronomy guidance in a small local knowledge library. The user types a question or uses an available speech-input route. The backend expands selected crop and problem terms, searches stored passages with PostgreSQL full-text search and returns up to three matching excerpts. The response includes document title, publisher, source URL and page number or section when available. A weak search match results in an abstention that asks for a more specific question or a local source of help.

The response contract is deliberately narrower than an open-ended chatbot. The text route does not ask Gemini or another LLM to retrieve documents, summarize them or compose advice. It returns stored passage text. The citation helps a user inspect the source behind the passage. Because the library has limited scope and English content, the system may fail to find relevant material or may retrieve a passage that is only partially relevant. It should not imply complete coverage.

## 6.2 HISTORY OF RAG AND THE CURRENT DESIGN

In an earlier project iteration, text generation using Gemini and a retrieval-augmented generation direction were included in planning. RAG is a useful research pattern in which retrieved documents are supplied to a language model as grounding context before generation. That approach would require evaluating retrieval relevance and answer faithfulness, including whether the generated response adds unsupported information. The current implementation has changed to deterministic retrieval-and-return. RAG remains in the report because it explains the feature's evolution, but it is not the current text answer system.

A separate optional speech transcription feature can use a Gemini API key on the server side. It converts speech to text and is not used to search the knowledge library or formulate text answers. Browser speech input/local text-to-speech may also be available independently. Treating transcription as a separate capability avoids confusing a speech-to-text dependency with text-advisor logic.

*Figure 6.1. Current Module 3 retrieve-and-return architecture; prior RAG direction shown separately.*

## 6.3 QUERY EXPANSION AND LANGUAGE COVERAGE

A Python synonym dictionary expands common crop and problem terms before full-text search. It recognizes selected English terms and selected Hindi, Tamil and Telugu terms. Expansion is explicit, inspectable and deterministic: maintainers can test a mapping and see which words it adds. It is not a machine translation system, language model or comprehensive linguistic analyzer. An unseen spelling, dialect term or multiword phrase may not expand. The returned passages remain in English, so query-term recognition does not mean the complete answer is localized.

The system uses PostgreSQL's simple text-search configuration. It does not apply language-specific stemming and does not perform semantic embedding search. The stored vector column and pgvector image are database compatibility elements; the active route uses PostgreSQL full-text search. Exact match quality depends on token overlap, indexed text, synonym coverage, ranking rules and corpus content. The ranking value is not calibrated to a probability that the passage answers the user's question.

## 6.4 KNOWLEDGE LIBRARY AND SOURCE PROVENANCE

The starter library contains six reviewed, paraphrased entries from ICAR and MANAGE source materials. The entries cover selected topics such as tomato canopy/blight context, potato late blight symptoms, maize disease advisories, grape black rot, integrated pest management and soil/nutrient management. These are not full copied PDFs. Document records keep source title, publisher, URL, reuse/licence note and checksum. Passage records store text, tags and a page or section when verified. The response attaches citations to the returned chunks.

Source review is part of the software workflow. A new source should be authoritative, relevant to the intended region, current enough for the claim and reusable under its stated terms. PDF text extraction must be reviewed for page ordering, table structure and printed-to-PDF page offsets. The corpus manifest should document publisher, URL, publication date, license/reuse status, topic tags and passage locations. Source content should be paraphrased carefully and quoted only where permitted.

| Entity | Purpose | Important fields |
|---|---|---|
| knowledge_documents | One source item and provenance record. | Title, publisher, source URL, licence note, SHA-256 checksum. |
| knowledge_chunks | Passage-level content available | Document ID, page, passage |

| | | |
|---|---|---|
| | to retrieval. | text, tags/metadata, indexed search vector. |
| advisor_messages | Conversation and retrieval event record. | Question, response/retrieved text, method, latency, abstention. |
| advisor_citations | Support relation between a response and passages. | Message ID, chunk/document reference and citation metadata. |

## 6.5 RESPONSE SAFETY AND ABSTENTION

The advisor avoids inventing an answer when there is no relevant passage. Weak matches lead to an abstention. Requests for fertilizer or pesticide dosage are blocked because quantitative application advice requires a current product label, local agronomic context, soil test and applicable regulations. The retrieved passages also include cautious statements that visual symptoms alone cannot confirm a disease and that local KVK or plant-pathology confirmation may be required.

Abstention is a safety behavior, but it can be wrong in two directions. The system may abstain even when a relevant passage exists, or it may return an apparently strong match that does not resolve the question. The current score is a ranking signal rather than a calibrated confidence. A gold query set should include questions with relevant entries, paraphrased questions, ambiguous questions, unsupported topics and hazardous dosage requests to test both retrieval and refusal behavior.

## 6.6 DATABASE AND MIGRATION REQUIREMENTS

The active advisor query expects a generated/searchable search_vector field on knowledge_chunks. The configured database used for the saved retrieval benchmark did not contain that column; the query failed with an undefined-column error before relevance metrics could be computed. The application launch scripts apply Alembic migrations and seed the knowledge library, but a stale database still needs its schema upgraded. A deployment checklist should record the current migration head and verify the search index and seed document count before declaring the advisor ready.

The failure does not mean the retrieval algorithm has a known low score. It means the evaluation could not run. The report therefore gives no precision@k, recall@k, MRR, nDCG, citation correctness or latency figure for the advisor. Once the database is aligned, the evaluation should log query set version, ground-truth passage IDs,

relevant-grade judgments, database version, query settings and exact corpus checksum.

## 6.7 FUTURE EVALUATION DESIGN

A practical first benchmark would use a small but carefully adjudicated query set from agronomy questions the corpus is meant to support. Each query should have one or more relevant chunk IDs, a topic label and an expected behavior: retrieve a source, abstain, or block. Evaluators should judge relevance independently, resolve disagreements, and keep the test questions out of synonym-dictionary tuning where possible. Results should report micro and macro scores by topic so that a common topic does not hide missing coverage in a less frequent one.

| Metric / check | Purpose for this advisor | Caution |
|---|---|---|
| Precision@k | Share of top-k results relevant to query. | A relevant chunk in top-k may not fully answer a multi-part question. |
| Recall@k | Share of known relevant passages retrieved in top-k. | Requires complete relevance judgments for each query. |
| MRR | Rank of the first relevant passage. | Does not measure relevance of later results. |
| nDCG@k | Ranking quality when relevance is graded. | Judgment rubric must be consistent and documented. |
| Citation validity | Returned source points to the stored passage/source location. | Valid link is not proof the advice is current or applicable. |
| Abstention accuracy | Appropriate refusal on unsupported/hazardous prompts. | Must separately score false abstention and unsafe retrieval. |
| Latency | Time from query receipt to response. | Report hardware, DB size and percentile. |

PASTE AUTHENTIC SCREENSHOT HERE

Figure 6.2. Module 3 retrieved passage and citation

Capture only after migration and database-backed retrieval succeed.

# CHAPTER 7

# IMPLEMENTATION AND DATA FLOW

## 7.1 FRONTEND COMPONENTS AND API BOUNDARY

The frontend is implemented in React with TypeScript and Vite. It presents module navigation and feature-specific components. Module 1 handles file selection, crop selection, upload progress, error feedback and the result/uncertainty state. Module 2 collects location, sowing date, water status and area, displays simple-engine shortlist cards and separates Random Forest and XGBoost tabs. Module 3 collects text or transcribed input and renders retrieved passages with citation details or an abstention.

The browser does not determine the model class or calculate the planner's scores locally. It serializes request fields and renders backend responses. This keeps validation and core logic centralized, but requires the UI to handle API failure, cancellation, stale requests, loading indicators and unexpected response shapes. The Module 2 review records a Cancel action, abort handling and stale-result protection so that a late response does not overwrite a cancelled lookup.

## 7.2 FASTAPI ROUTES AND REQUEST VALIDATION

FastAPI exposes module endpoints, including health/prediction, planning and advisor routes. Pydantic request models constrain inputs such as coordinates, area, date and irrigation status. Extra/unexpected fields are rejected where needed. File upload paths validate image metadata before calling model inference. Boolean values are not silently accepted as numeric coordinates or field area. The backend checks that requested dates are valid in the India calendar and that external weather responses have the expected date range and value order.

| Route family | Typical request data | Backend responsibility |
|---|---|---|
| Health scan | Image bytes and selected crop. | Validate file; run model; compare crop/confidence; store metadata. |
| Planning location search | Place-name query or coordinate selection. | Geocode/filter location; ensure selected result is explicit. |
| Planning recommendation | Coordinates, location name, sowing date, irrigation, area in hectares. | Fetch soil/weather; validate source data; screen and rank candidates. |
| Advisor chat | Question text and optional conversation context. | Expand terms; query PostgreSQL; return excerpts/citations or abstain. |
| Translation / speech support | Separate text or audio input, depending on route. | Optional speech/translation capability; not part of text retrieval response logic. |

## 7.3 MODULE 1 REQUEST SEQUENCE

The browser starts a scan only after the user chooses an image and crop. The request is sent to the backend as a multipart upload. The API validates file type, size and decoding, then applies leaf suitability checks. If the file is rejected, inference is skipped and a validation error is returned. If accepted, preprocessing prepares model input and the MobileNetV2 checkpoint produces a class-score vector. The backend selects the maximum class, computes the confidence display value and checks crop consistency. The service returns the result state and writes scan metadata with the file hash, not the original photo.

The request should be treated as an ephemeral processing transaction. Temporary byte buffers are needed to decode the uploaded file, but database persistence excludes the leaf photo. Observability should log request IDs, validation status and processing time without writing file bytes or unintended image content. A privacy audit should inspect both application logs and reverse-proxy logs because application-level retention statements do not control external log configuration.

## 7.4 MODULE 2 REQUEST SEQUENCE

The user searches a place or explicitly requests GPS. The geocoder may return multiple results; the user selects the intended location rather than relying on an ambiguous first result. The frontend submits the selected coordinates, sowing date, water availability and area. The backend retrieves the nearby soil map values, current/seven-day forecast and ten-year historical climate. It validates responses and applies the crop catalogue's eligibility conditions. The simple planner computes benchmark surplus for eligible candidates. If data are missing, the API communicates an insufficient or provisional state rather than inventing replacement values.

The experimental ML tabs use the same farmer context to form candidate crops after the simple screening stage. The model service estimates yield from its historical state/district/crop/year table; soil and weather values are not direct model features. The frontend then applies the same static price and cost references to model estimates for ranking. This makes the two engines visually comparable but does not make their inputs or prediction targets identical. The UI must preserve the label "experimental yield estimate" to avoid confusing it with the simple benchmark engine.

## 7.5 MODULE 3 REQUEST SEQUENCE

A typed question goes through tokenization and deterministic synonym expansion. The service constructs a PostgreSQL full-text query and searches indexed knowledge chunks. Ranking and rule checks select a small number of distinct passages. The backend returns the stored passage text and citation metadata. If the database is missing the expected indexed column, the request fails before retrieval. If the database is current but no relevant passage is found, the service returns an explicit abstention. These are different states and should have separate diagnostics.

## 7.6 DATA PERSISTENCE AND RETENTION

| Record | Stored information | Not stored / note |
|---|---|---|
| Scan metadata | Prediction, confidence/uncertainty, crop, image hash and metadata. | Uploaded leaf photo not saved as scan record. |
| Advisor message | Question, retrieved response/context, method, latency and abstention flag. | No generated answer text from Gemini in current text route. |
| Advisor citation | Link between advisor message and knowledge chunk/source. | Citation quality still requires a benchmark. |
| Knowledge document | Publisher, title, URL, checksum and reuse metadata. | Private or restricted documents should not be ingested without approval. |
| Knowledge chunk | Reviewed passage, page/section, tags and indexed representation. | Seed text is limited and English answer passages. |

## 7.7 ERROR STATES AND RECOVERY

The system encounters errors from invalid inputs, public-service failures, schema mismatch, cancellations and frontend component failures. Error handling should return an understandable state and avoid silent coercion. The Module 2 implementation retries one time for selected transient network errors and gateway responses, does not repeatedly retry quota failures, and validates that forecast dates are complete and temperature lows do not exceed highs. Soil results are cached only when complete so that a partial response does not prevent later recovery.

The frontend polls backend health while active and reports failure/recovery rather than leaving a stale “connected” indicator. Requests can be cancelled and late responses are ignored. An error boundary can catch rendering failures and offer a reload path. These behaviors improve user experience but do not establish agronomic

validity. Integration tests should assert the exact message and data state for each failure, not only that an HTTP request completes.

## 7.8 DEPLOYMENT AND CONFIGURATION

Docker Compose starts PostgreSQL using a pgvector image. The frontend and backend are run as separate local processes by the start script rather than a single all-in-one Compose service. Alembic applies schema changes when the application starts. Optional environment keys support separate services such as Gemini speech transcription; the key must remain server-side. Public data requests are subject to provider availability, rate limits, attribution and terms. Before commercial or public deployment, service plans and licensing need to be reviewed.

A clean deployment should record the application commit, Python/Node package lock versions, database migration head, model checkpoint checksum, yield CSV checksum, crop catalog version, source reference date and runtime hardware. Without this information, screenshots and latency scores are difficult to reproduce. The project's current evaluation package records data split and model metrics for Module 2 and an earlier split summary for Module 1, but not every environmental dependency is frozen in this report.

# CHAPTER 8

# TESTING AND EVALUATION

## 8.1 TESTING LEVELS AND PURPOSE

Software tests, model evaluation and field validation answer different questions. Unit tests check a specific rule or function. Integration tests check that multiple services or components work together. End-to-end tests exercise a complete user workflow. Model metrics estimate predictive performance on a defined labeled test set. Field trials measure behavior under real use and agronomic conditions. A high count of passing software tests does not imply that a model is accurate, and a high model accuracy does not prove that the interface or database is reliable.

For this report, the evidence is grouped by type. Module 1 has saved classification metrics but could not be rerun because test images were missing. Module 2 has a

saved offline RF/XGBoost evaluation on a common chronological holdout and recorded software test results for the planning workflow. Module 3 has implementation evidence but no retrieval-quality benchmark because the local database schema did not match. There is no multi-season farmer field trial for any module.

*Figure 8.1. The evaluation target and metrics differ by module; missing evidence is called out.*

## 8.2 CLASSIFICATION METRICS

For a multiclass classifier, accuracy is the number of correct predictions divided by the number of test samples. Precision for a class asks how many predictions of that class were correct. Recall asks how many true examples of that class were found. F1 is the harmonic mean of precision and recall. Macro averaging computes a metric for every class and gives each class equal weight; weighted averaging weights each class by support. A confusion matrix shows how often each true class is assigned to each predicted class.

Latency is separate from predictive quality. Median (p50) inference latency describes the midpoint of measured request times, while p95 shows a tail threshold. Latency depends on machine, framework, preprocessing, thread settings, warm-up and whether network/API overhead is included. The saved Module 1 p50 and p95 figures are inference timing records and should not be described as end-to-end application response times unless the timing definition confirms that scope.

| Metric | Definition | Use in this project | Known limit |
|---|---|---|---|
| Accuracy | Correct / all test cases. | Overall ten-class classification summary. | Can hide weak classes. |
| Precision | TP / (TP + FP), per class. | False positive balance. | Depends on class distribution and decision threshold. |
| Recall | TP / (TP + FN), per class. | Missed disease-class examples. | Does not capture cost/severity of different errors. |
| F1 | Harmonic mean of precision and recall. | Single classwise balance metric. | Not a substitute for confusion matrix. |
| Macro average | Unweighted average across classes. | Equal class importance. | Small-support classes have noisy estimates. |
| p50 / p95 | Median / 95th percentile measured latency. | Runtime experience. | Hardware and timing scope specific. |

## 8.3 MOBILEV2 SAVED CLASSIFICATION RESULTS

The saved evaluation artifact reports 1,591 test samples and the following aggregate metrics. Because the test images were absent in the current project folder, these numbers are reproduced from stored report files, not a new model execution. The report should preserve that qualification in any slide, poster or viva presentation. A screenshot of the saved evaluation matrix may be included as documentary evidence, but it should be labeled “saved artifact” unless the rerun is completed. [22]

| Model | Dataset/test split | N | Accuracy | Macro precision | Macro recall | Macro F1 |
|---|---|---|---|---|---|---|
| MobileNetV2 | PlantVillage-derived 10-class saved test | 1,591 | 98.6801% | 98.9150% | 98.3695% | 98.6264% |

*Table 8.1. Stored Module 1 aggregate metrics; not independently recomputed during report preparation.*

## 8.4 REGRESSION METRICS AND BASELINE

The Module 2 offline experiment compares predicted district yield to observed yield. MAE is the average absolute error, in kg/ha; lower is better. RMSE is the square root of average squared error, also in kg/ha; it penalizes larger errors more strongly than MAE. $R^2$ compares model error with a reference prediction of the test-set mean; values closer to one indicate stronger fit relative to this baseline, while negative values indicate worse performance than the mean under the defined calculation. Runtime records are specific to the machine used for the evaluation. [23]

A training-mean baseline is necessary because a model score should be interpreted relative to a simple predictor. The mean baseline achieved MAE 770.31 kg/ha, RMSE 1,252.26 kg/ha and $R^2$ −0.2075. Random Forest and XGBoost reduce error on this split, but their $R^2$ values are only about 0.1. This means the models explain only a limited fraction of the held-out variation by this metric and configuration. It is a meaningful negative result: the experimental pipeline works, yet the available features and validation design are not sufficient evidence for a strong crop-yield forecast.

| Model | MAE kg/ha ↓ | RMSE kg/ha ↓ | $R^2$ ↑ | Fit (s) | Predict (s) |
|---|---|---|---|---|---|
| Training-mean baseline | 770.31 | 1,252.26 | −0.2075 | 0.13 | 0.01 |
| Random Forest | 676.89 | 1,071.50 | 0.1159 | 35.037 | 0.061 |

| | | | | | |
|---|---|---|---|---|---|
| XGBoost | 683.10 | 1,080.97 | 0.1003 | 0.404 | 0.020 |

*Table 8.2. Saved final-run temporal evaluation. Lower MAE/RMSE and higher R² are preferred.*

## 8.5 TEMPORAL SPLIT AND EXPERIMENT PROTOCOL

The test split holds out the most recent five years available in the dataset: 2013–2017. Training rows range from 1966–2012. The same 10,011 test rows are evaluated for the baseline, Random Forest and XGBoost. The training portion has 89,651 rows. There are 304 districts represented in both partitions. The split is chronological, which helps avoid training on later years and testing on earlier years, but district overlap remains. A strict new-district test would hold out entire districts; a stronger forward test would repeat training and testing across multiple temporal cutoffs.

Offline features are state, district, year and crop. The target is yield in kg/ha. Categorical feature encoding differs from the live service: the offline comparison represents year categorically, while the user-interface service uses numeric year and per-crop sample caps. The offline models use 120 trees/estimators; the UI models use 60. These settings must be reported whenever a score is quoted. Fixed hyperparameters simplify the initial comparison but do not represent a tuned best model.

## 8.6 RANDOM FOREST AND XGBOOST RESULT INTERPRETATION

Random Forest has MAE 6.21 kg/ha lower than XGBoost and RMSE 9.46 kg/ha lower on the shared holdout; its R² is higher by about 0.0157. Those are small relative differences in the context of errors above 1,000 kg/ha. The result supports a narrow statement: RF did slightly better on the specified metrics for this run and split. It does not support the general statement that Random Forest is always more trustworthy or will perform better on another district, crop or year.

XGBoost fit in about 0.404 seconds versus about 35.037 seconds for RF in the recorded environment, and predicted in about 0.020 versus 0.061 seconds. This runtime difference may reflect implementation details, training setup and hardware. Runtime should be reproduced in a controlled environment before it is used as an algorithmic advantage. A better comparison would repeat training across seeds, tune hyperparameters inside training years, report bootstrap confidence intervals and test

additional baselines such as crop-specific historical averages or gradient boosting variants.

## 8.7 SOFTWARE TESTS AND PLANNER INTEGRATION

The Module 2 code review records 87 passing backend tests, including 27 planning tests, and 37 passing frontend tests. TypeScript checks and a Vite production build passed. Public-data smoke tests were conducted for Warangal, Ludhiana and Coimbatore. Tests cover economic units, crop ranking, seasonal yields, water constraints, missing/invalid data, location filtering, request validation, area conversion, absence of automatic location collection, cancellation and stale response behavior. These counts demonstrate implementation coverage, not predictive accuracy or agronomic validation.

| Evidence category | Recorded result | Valid interpretation |
|---|---|---|
| Backend tests | 87 passed; 27 planning-focused. | Automated code paths passed the recorded test suite. |
| Frontend tests | 37 passed. | Tested frontend behavior passed the recorded suite. |
| Build checks | TypeScript and Vite production build passed. | Frontend build completed successfully at review time. |
| Live smoke checks | Warangal, Ludhiana, Coimbatore. | Public provider integration exercised; not field endorsements. |
| Field performance | No prospective field trial reported. | No real-world recommendation accuracy claim. |

## 8.8 MODULE 3 EVALUATION STATUS

The advisor retrieval benchmark could not complete in the configured local environment because the expected knowledge_chunks.search_vector column was absent. Consequently, there is no measured precision@k, recall@k, mean reciprocal rank, nDCG, citation accuracy, abstention accuracy or latency table for Module 3. The report should not fill this gap by describing code-level unit tests as retrieval performance. Once migrations are applied, a versioned gold query set and evaluation script are required.

## 8.9 EVALUATION SUMMARY MATRIX

| Module / track | Task | Available score | Evidence state |
|---|---|---|---|
| 1 · Crop Health | Ten-class image classification. | 98.68% accuracy; macro F1 98.63%; N=1,591. | Saved metric artifact; not rerun. |
| 2 · Simple planner | Rules and benchmark-based | No classification/regression | Software tests and public-source smoke |

| | | | |
|---|---|---|---|
| | screening/ranking. | score. | checks. |
| 2 · RF/XGBoost | District yield regression. | RF MAE 676.89, RMSE 1,071.50, $R^2$ .116; XGB MAE 683.10, RMSE 1,080.97, $R^2$ .100. | Common temporal holdout; local limitation. |
| 3 · Advisor | Passage retrieval and citation. | No retrieval-quality score. | Benchmark blocked by local schema mismatch. |

CHAPTER 9

# RESULTS, LITERATURE COMPARISON AND DISCUSSION

## 9.1 COMPARISON PRINCIPLES

Research comparisons are credible only when readers can understand whether the studies solved the same task. The algorithm name alone is not enough. A comparison must identify target variable, dataset, class count or crop set, geographic coverage, years, observation unit, train/test split, input features, metric units and whether the result is cross-validated, held out or in-sample. If these differ, reported scores may be listed side by side to show context, but they should not be ranked as if the experiments were identical.

This principle is especially important here. Module 1 classifies ten labels, whereas several papers classify 38 or 39 labels. Module 2's saved machine-learning result predicts yield in kg/ha, while crop-recommendation papers often classify a crop name. One cited yield paper reports errors on a different scale, and another uses rice-specific climate and remote-sensing features. The report therefore distinguishes contextual literature matrices from the one direct internal comparison where all algorithms share the same test rows.

## 9.2 CROP HEALTH RESEARCH COMPARISON

| Study | Architecture / task | Dataset and protocol | Published / saved result | Comparison note |
|---|---|---|---|---|
| KrishiMitra | MobileNetV2; 10-label classification. | PlantVillage-derived ten-class held-out split; N=1,591. | Saved accuracy 98.68%; macro F1 98.63%. | Not rerun because images missing; not field validated. |
| Mohanty et al. (2016) [3] | CNN; plant/disease classification. | 54,306 controlled PlantVillage images; 38 classes. | Accuracy 99.35%. | Different label space and split; controlled |

| | | | | |
|---|---|---|---|---|
| | | | | benchmark. |
| Hukkeri et al. (2024) [4] | Pretrained CNN comparison; MobileNetV2. | PlantVillage; 38 classes. | MobileNetV2 accuracy 93.5%. | Related architecture; protocol differs. |
| Duhan et al. (2025), baseline [5] | MobileNetV2 on augmented PlantVillage. | 39 classes; study-specific train/test. | Accuracy 99.87%. | Different split and training. |
| Duhan et al. (2025), proposed [5] | RTR_Lite_Mobile NetV2 with attention. | 39 classes; same paper experiment. | Accuracy 99.92%. | Different architecture and protocol. |
| Singh et al. (2020), PlantDoc | Visual disease dataset. | 2,598 web images; 13 species, up to 17 classes. | Dataset contribution. | Potential external-domain benchmark after label mapping. |

The project's saved accuracy lies below the 2016 and 2025 PlantVillage scores and above the supplied 2024 MobileNetV2 score. That ordering is descriptive only. It cannot be used to claim the project is better than one paper or worse than another, because image partitions, class definitions, trainable layers, augmentations and evaluation choices differ. A stronger result would reproduce chosen baselines on one common split and evaluate them on a separate field set.

## 9.3 CROP RECOMMENDATION AND YIELD STUDY COMPARISON

| Study | Prediction target / scope | Reported metrics | Direct comparison? |
|---|---|---|---|
| KrishiMitra RF | District yield, kg/ha; 2013–17 holdout. | MAE 676.89; RMSE 1,071.50; $R^2$ .1159. | Yes vs local XGBoost/baseline only. |
| KrishiMitra XGBoost | Same yield table and test rows. | MAE 683.10; RMSE 1,080.97; $R^2$ .1003. | Yes vs local RF/baseline. |
| Thihlum et al. (2025) [21] | Crop-label recommendation in three Mizoram districts. | Accuracy/P/R/F1 approximately .98. | No; classification versus regression. |
| Arumugam et al. (2024) | Crop recommendation using environment and ML methods. | No complete auditable numeric comparison table. | No numeric comparison. |
| Jhajharia et al. (2023) [11] | Five-crop yield prediction in Rajasthan. | RF $R^2$ .963; RMSE .035; MAE .0251. | No; scale, features and split differ. |
| De Clercq & Mahdi (2024) [12] | Rice yield across Indian districts, weather/remote sensing. | Best $R^2$ .82; MAE .29; MAPE .16. | No; crop, target scale and features differ. |

The RF-PSO-XGBoost result in the Mizoram paper is a classification score; the output is a crop label. KrishiMitra's offline RF/XGBoost test predicts continuous yield. Accuracy and MAE do not measure the same quantity and cannot be compared numerically. A 0.98 classifier accuracy does not mean that a yield model with $R^2$ 0.116 is inferior by the same scale. Likewise, an $R^2$ of 0.963 in Rajasthan cannot be interpreted as a direct improvement without checking its target transformation, split, crop subset and feature inputs.

## 9.4 WHY PROJECT SCORES DIFFER FROM PAPERS

Dataset size and variation matter. A dataset with fewer regions or more uniform conditions can be easier to predict than a broad historical table. A model may also appear stronger if the test set contains near-duplicate examples, data from the same district/year pattern as training, or features that indirectly reveal the target. The project's chronological split reduces one form of leakage by training on earlier years, but retains districts across the split and lacks environmental variables. It is a useful but incomplete forecasting design.

Feature richness also differs. The Indian rice-yield study uses climate and earth-observation predictors designed to capture seasonal crop conditions. The project's offline model uses only state, district, crop and year. Those identifiers can encode historical averages, but they cannot explain whether a particular future season is wetter, hotter or better managed. The low $R^2$ can reveal a real data limitation rather than merely a weak algorithm.

Metric reporting choices matter. Regression MAE and RMSE depend on target units and transformations; $R^2$ depends on target variance and evaluated sample. Classification accuracy depends on class balance and label definitions. Latency depends on hardware and timing boundaries. Each value must be paired with its task and context. The matrices show these differences instead of claiming "better" from an isolated number.

## 9.5 THREATS TO VALIDITY

| Threat | Effect on this report | Next evidence needed |
|---|---|---|
| Missing Module 1 images | Saved scores cannot be recomputed. | Restore image files and rerun. |
| PlantVillage domain shift | Controlled metrics may not transfer to field photos. | External field set with specialist labels. |
| Uneven class support | Small class metric estimates are uncertain. | Larger test set and intervals. |
| Historical yield features | No soil/weather/management inputs; low $R^2$. | Matched environmental and seasonal data. |
| District overlap | No test of unseen district transfer. | Spatially held-out test. |
| Old yield data | Records end in 2017. | Newer audited observations. |
| UI/offline model difference | Offline metrics may be mistaken as UI accuracy. | Separate model cards and versioned configs. |
| Different paper tasks | Classification and yield scores are incomparable. | Common benchmark or cautious context. |
| Advisor schema failure | No retrieval score was measured. | Migrate and run gold query set. |
| No field trial | No user outcome evidence. | Prospective local pilot. |

## 9.6 CLAIMS SUPPORTED BY PRESENT EVIDENCE

| Evidence supports | Evidence does not support |
|---|---|
| Saved MobileNet report lists 98.68% accuracy on a 1,591-image ten-class split. | Current field diagnosis accuracy is 98.68%. |
| RF slightly outperformed XGBoost on one common local temporal test. | RF is always more trustworthy than XGBoost. |
| Both yield regressors improved on the mean baseline in this run. | The model identifies the farmer's most profitable crop. |
| Simple planner ranks supported crops with rules and national benchmarks. | Benchmark surplus is guaranteed net profit. |
| Advisor code retrieves stored passages and citations. | Current text advisor uses Gemini or has a measured high answer score. |
| Configured database blocked Module 3 evaluation. | Module 3 relevance has been quantitatively validated. |

## 9.7 IMPLICATIONS FOR PROJECT CREDIBILITY

Credibility improves when a report states exactly what was evaluated, includes a baseline, reports modest results, preserves a reproducible configuration and separates implementation evidence from published claims. The RF/XGBoost matrix is useful because both models share the same test rows. It is not evidence of crop-choice quality because the target is yield and the observations are historical districts. A stronger comparison could reproduce selected recommendation papers on a common dataset or evaluate the project on an externally validated field decision dataset.

Academic critique should not select studies because their scores are favorable or their methods appear weak. A fair review identifies protocol limitations with evidence and applies the same standards to KrishiMitra. The project's own limitations—saved rather than rerun image metrics, low yield $R^2$, old data, district overlap and absent retrieval benchmark—must be disclosed beside study differences.

# CHAPTER 10

# CONCLUSION AND FUTURE WORK

## 10.1 CONCLUSION

KrishiMitra has evolved into a modular prototype with three separate information workflows. Module 1 uses MobileNetV2 to screen supported leaf images and indicate uncertainty. Module 2 combines public soil/weather information and crop-range rules to filter a nine-crop catalogue, then ranks passing candidates with national yield, price

and cost references. A separate experimental path compares Random Forest and XGBoost yield estimates. Module 3 retrieves stored agronomy passages through PostgreSQL full-text search and returns source metadata. RAG is retained as project history, while current text answers are retrieved passages rather than model-generated advice.

The strongest current quantitative claim is narrow: in the saved chronological district-yield test, Random Forest had slightly lower error than XGBoost and both improved on a mean baseline. Their low $R^2$ values show that the experimental models remain limited. MobileNetV2's saved metrics are high on a controlled ten-class split, but the images were absent for a fresh rerun and no field validation is available. The Module 3 quality benchmark was blocked by a local schema mismatch. This report therefore does not establish that KrishiMitra outperforms published methods or predicts an individual farmer's profit.

The revised report follows the submitted template sequence and written formatting rules, with academic front matter, chapters, conclusion/future work, appendices and APA references. New diagrams make module boundaries and data paths more legible. Screenshot placeholders are reserved for authentic frontend and evaluation evidence. They should be replaced only after the respective test or run has actually been captured.

## 10.2 FUTURE WORK PRIORITIES

- Restore the exact MobileNetV2 checkpoint and test images; reproduce the metrics and archive the split manifest, class map, software versions and confusion matrix.
- Collect independent field images with farmer consent and specialist labels across lighting, cameras, regions, cultivars and symptom stages.
- Add calibration and selective prediction evaluation so uncertainty behavior is measured instead of relying only on a fixed 75% score threshold.
- Join district-season-year yield observations with weather, soil, crop calendar and market/cost data using documented spatial and temporal alignment.
- Use rolling-origin and held-out-district evaluation; tune only within training periods and report repeated runs and intervals.

- Choose the actual model target: yield, crop suitability, gross margin or risk-adjusted profit. Avoid describing yield regression as profit prediction.
- Apply Alembic migrations and seed the advisor. Evaluate on human-adjudicated queries using precision@k, recall@k, MRR/nDCG, citation checks and abstention measures.
- Expand the curated knowledge library with current regional sources and native-speaker reviewed Hindi, Tamil and Telugu responses.
- Validate soil map estimates against permitted field samples and update local price/cost references with provenance and licensing.
- Conduct a local pilot study with preregistered endpoints, user consent and agronomist oversight before making adoption claims.

## 10.3 FINAL POSITION

The current deliverable is best positioned as an integrated agricultural decision-support prototype with an experimental model-comparison component. It demonstrates how image classification, rule-based planning, tabular yield regression and passage retrieval can coexist while maintaining separate evidence and limitations. The next research contribution should evaluate the user-facing decision on matched, reproducible data rather than claim universal superiority from cross-paper metric values.

# REFERENCES

References are numbered in the order they first appear. Each bracketed citation corresponds to one numbered entry below.


[1] Department of Agriculture and Farmers Welfare. (2025). Agricultural statistics at a glance 2024–25. Government of India. https://nccd.gov.in/uploads/Agricultural_Statisitcs_at_a_Glance_2024_25_11da3cd37d.pdf

[2] PostgreSQL Global Development Group. (2026). Full text search. PostgreSQL documentation. https://www.postgresql.org/docs/current/textsearch.html

[3] Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using deep learning for image-based plant disease detection. Frontiers in Plant Science, 7, 1419. https://doi.org/10.3389/fpls.2016.01419

[4] Hukkeri, S., Soundarya, S., Gururaj, G., & Ravi, R. (2024). Classification of various plant leaf disease using pretrained convolutional neural network on ImageNet. The Open Agriculture Journal, 18. https://doi.org/10.2174/0118743315305194240408034912

[5] Duhan, S., Gulia, P., Gill, N. S., & Narwal, E. (2025). RTR_Lite_MobileNetV2: A lightweight and efficient model for plant disease detection and classification. Current Plant Biology, 42, 100459. https://doi.org/10.1016/j.cpb.2025.100459

[6] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 4510–4520). https://openaccess.thecvf.com/content_cvpr_2018/html/Sandler_MobileNetV2_Inverted_Residuals_CVPR_2018_paper.html

[7] Singh, D., Jain, N., Jain, P., Kayal, P., Kumawat, S., & Batra, N. (2020). PlantDoc: A dataset for visual plant disease detection. In Proceedings of the 7th ACM IKDD CoDS and 25th COMAD (pp. 249–253). https://doi.org/10.1145/3371158.3371196

[8] Breiman, L. (2001). Random forests. Machine Learning, 45, 5–32. https://doi.org/10.1023/A:1010933404324

[9] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785–794). https://doi.org/10.1145/2939672.2939785

[10] XGBoost developers. (2026). XGBoost documentation. https://xgboost.readthedocs.io/en/stable/

[11] Jhajharia, A., Mathur, A., Jain, P., & Nijhawan, S. (2023). Crop yield prediction using machine learning and deep learning techniques. Procedia Computer Science, 218, 406–417. https://doi.org/10.1016/j.procs.2023.01.023

[12] De Clercq, D., & Mahdi, A. (2024). Feasibility of machine learning-based rice yield prediction in India at the district level using climate reanalysis and remote sensing data. Agricultural Systems, 220, 104099. https://doi.org/10.1016/j.agsy.2024.104099

[13] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33. https://arxiv.org/abs/2005.11401

[14] OpenLandMap. (n.d.). Global soil property maps and project documentation. https://docs.openlandmap.org/

[15] Open-Meteo. (n.d.). Weather forecast API documentation. https://open-meteo.com/en/docs

[16] Open-Meteo. (n.d.). Historical weather API documentation. https://open-meteo.com/en/docs/historical-weather-api

[17] Food and Agriculture Organization of the United Nations. (n.d.). ECOCROP database. https://www.fao.org/land-water/resources/tools/databases/ecocrop/en

[18] Government of India, Press Information Bureau. (2026). Cabinet approves minimum support prices for Kharif crops for marketing season 2026–27. https://www.pib.gov.in/PressReleasePage.aspx?PRID=2260617

[19] Government of India. (2026). Minimum support prices for Rabi crops for marketing season 2027–28. Press Information Bureau. https://www.pib.gov.in/newsite/erelcontent.aspx?lang=2&reg=3&relid=295517

[20] TCI Cornell. (n.d.). Data. https://tci.cornell.edu/data/

[21] Thihlum, Z., Ambeth Kumar, V. D., & Mohanty, A. K. (2025). Explainable XGBoost model for crop recommendation in Mizoram using hybrid random forest and particle swarm optimization. The Indian Journal of Agricultural Sciences, 95(11), 1324–1331. https://doi.org/10.56093/ijas.v95i11.166636

[22] KrishiMitra development team. (2026). KrishiMitra project source code, module documentation and saved evaluation artifacts [Unpublished project materials].

[23] scikit-learn developers. (2026). Model evaluation: Quantifying the quality of predictions. scikit-learn documentation. https://scikit-learn.org/stable/modules/model_evaluation.html

# APPENDICES

## APPENDIX A. DATA DICTIONARIES AND SOURCE FIELDS

### A.1 CROP-YIELD REGRESSION INPUTS

The offline regression table contains a row for a crop observation at a district and year. It is a historical label table, not a table of current farm conditions. Its geographic identifiers and calendar year are categorical or temporal descriptors. The target is observed yield. No row directly identifies farmer-entered area, current soil map values, sowing date, live weather or realized farmer margin.

| Field | Role | Type / unit | Use in offline model | Caveat |
|---|---|---|---|---|
| state | Predictor | Categorical name | Geographic identity. | Historical state names may change/normalize. |
| district | Predictor | Categorical name | District identity. | Boundaries/names may vary over long period. |
| year | Predictor | Year integer/category | Temporal trend. | Offline encoded categorical; UI uses numeric year. |
| crop | Predictor | Categorical name/variant | Crop identity. | Only supported variants survive filtering. |
| yield | Target | kg/ha | Regression outcome. | Historical district yield, not field yield. |
| production | Source-side field if present | tonnes | Not included in the stated RF/XGB feature set. | Must not accidentally leak into predictors. |
| area | Source-side field if present | hectares | Not included in stated predictor set. | Can be post-harvest / target-related. |

### A.2 SIMPLE PLANNER DATA

| Variable | Source / derived from | Unit / representation | Current role |
|---|---|---|---|
| latitude / longitude | Selected geocoder result or explicit GPS. | Decimal degrees. | Soil/weather cell and forecast lookup. |
| location_name | User-selected geocoder label. | Text. | Display and request metadata. |
| sowing_date | Farmer input. | ISO calendar date. | Sowing-window and forecast overlap checks. |
| irrigation | Farmer input. | Unknown / rainfed / available. | Water screening and cautions. |
| area_ha | Farmer area and unit conversion. | Hectares. | Scales benchmark surplus. |
| soil_pH | OpenLandMap raster value conversion. | pH estimate. | Check against absolute crop range. |
| organic_carbon | OpenLandMap raster estimate. | Percent after conversion. | Displayed only; no eligibility use. |

| | | | |
|---|---|---|---|
| sand_fraction | OpenLandMap raster estimate. | Percent. | Displayed only; no eligibility use. |
| historical temperature | ERA5 daily values aggregated by month. | °C. | Four-month crop-temperature check. |
| annual rainfall | ERA5 historical precipitation aggregation. | mm/year. | Crop rainfall range check. |
| forecast | Open-Meteo forecast API. | Daily temperature/rain. | Near-term sowing caution. |
| yield_ref / price / cost | Government benchmark catalogue. | kg/ha, ₹/quintal. | Simple-engine benchmark surplus. |

### A.3 ADVISOR SOURCE AND PASSAGE METADATA

Knowledge document metadata provides provenance and supports citation rendering. A document has a title, publisher, source URL, a reuse/license note where available, and a checksum. A passage stores the selected text, page number when verified, section label and tags. The current seed passages are paraphrased excerpts; the source link should let users inspect the original context. If pagination differs between a PDF viewer and the printed page, the manifest should explain the offset.

| Record field | Meaning | Quality check |
|---|---|---|
| source_title | Display title for cited document. | Match publisher title/page. |
| publisher | Issuing organization. | Prefer primary/source authority. |
| source_url | Link to original source. | HTTPS, accessible, stable where possible. |
| licence_note | Reuse or redistribution limitation. | Review before storing text. |
| sha256 | Checksum of source artifact. | Recompute if source file changes. |
| page_number | Printed or verified PDF page. | Confirm offset, do not guess. |
| section | Named section/advisory. | Use a heading present in source. |
| chunk_text | Paraphrased or permitted passage. | Review for context and claim boundaries. |
| tags | Search topics and synonyms. | Controlled vocabulary and reviewed expansion. |

## APPENDIX B. EVALUATION METRICS AND WORKED INTERPRETATIONS

### B.1 MULTICLASS CONFUSION COUNTS

For one class treated as positive versus all other classes, true positive (TP) is a correct prediction of the class, false positive (FP) is a different true class predicted as this

class, false negative (FN) is this class predicted as something else, and true negative (TN) is any non-class example correctly assigned elsewhere. Precision = TP/(TP+FP); recall = TP/(TP+FN); F1 = 2 × precision × recall/(precision + recall). In a multiclass setting, these values are calculated per class from a confusion matrix, then macro averages are computed as the arithmetic mean across classes.

Overall accuracy = number of correct predictions divided by total examples. A class with very few test examples can have a perfect score by chance. For that reason, support must be included with class metrics, and a confusion matrix should be inspected to identify consistent confusions. Macro F1 gives low-support classes equal weight but does not make their estimates statistically precise. Confidence intervals or repeated evaluation samples would help convey uncertainty.

### B.2 REGRESSION ERROR METRICS

For observed yields $y_i$ and predictions $\hat{y}_i$ over n records, mean absolute error is MAE = $(1/n)\ \Sigma\ |y_i - \hat{y}_i|$. Root mean squared error is RMSE = $\text{sqrt}[(1/n)\ \Sigma\ (y_i - \hat{y}_i)^2]$. $R^2 = 1 - [\Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2]$, where $\bar{y}$ is the mean observed test target in the metric calculation. MAE is interpretable as the typical absolute deviation in the target's units. RMSE gives larger errors greater weight. $R^2$ is relative to variation around the test mean and may be negative when predictions are poor.

The reported MAE of 676.89 kg/ha for Random Forest means that the absolute difference between predicted and observed yield averaged about 677 kg/ha across the 10,011 test rows. It does not mean every prediction is within 677 kg/ha, nor does it indicate whether error is larger for a specific crop or district. The RMSE of 1,071.50 kg/ha exceeds MAE, suggesting that larger deviations have a greater effect on the squared-error average. The $R^2$ of 0.1159 indicates only a modest reduction in squared error relative to the mean reference under this evaluation design.

### B.3 REGRESSION BASELINE COMPARISON

| Metric | Mean baseline | Random Forest | XGBoost | Best in this run |
|---|---|---|---|---|
| MAE kg/ha | 770.31 | 676.89 | 683.10 | Random Forest |
| RMSE kg/ha | 1,252.26 | 1,071.50 | 1,080.97 | Random Forest |
| $R^2$ | −0.2075 | 0.1159 | 0.1003 | Random Forest |
| Fit time seconds | 0.13 | 35.037 | 0.404 | XGBoost by recorded time |
| Predict seconds | 0.01 | 0.061 | 0.020 | XGBoost by recorded time |

“Best in this run” is local to the saved machine, fixed parameters, feature set and split. It does not support a general ordering. A robust comparison should tune models on a validation window that precedes the final holdout, evaluate at multiple future cutoffs, repeat stochastic fits, and report uncertainty in both errors and model differences. If the project later changes the UI configuration, those new models require a separate score table.

### B.4 RETRIEVAL METRIC DEFINITIONS FOR A FUTURE TEST

For a query q, let the retrieved list at rank k be $R_k$ and the human-judged relevant set be G. Precision@k = $|R_k \cap G|/k$. Recall@k = $|R_k \cap G|/|G|$. Reciprocal rank is 1/r where r is the rank of the first relevant passage, or zero if none is found; mean reciprocal rank averages this value across queries. Normalized discounted cumulative gain uses graded relevance values and discounts lower-ranked results. Citation validity checks whether a citation resolves to the source and passage used. These metrics require query-specific judgments and a versioned corpus.

Retrieval quality should be reported by topic, language of the query, supported/unsupported status and hazard category. An overall average may hide that potato questions are answered reliably while soil or pesticide questions are not covered. Abstention evaluation should report false abstention on answerable questions and unsafe retrieval on unsupported or hazardous questions. A correct citation link is necessary for traceability but does not establish that the quoted guidance is current or suitable for the user’s location.

## APPENDIX C. SOFTWARE TEST CATALOGUE AND EVALUATION CASES

The catalogue below summarizes behaviors documented by the repository review and the evaluation plan. It is not a new test run. The project review reports 87 passing backend tests, including 27 planning-focused tests, and 37 passing frontend tests. “Expected” describes designed behavior; it does not mean every manual case was separately executed during this report update.

| ID | Area | Test condition | Expected result / evidence |
|---|---|---|---|
| C-01 | Image input | Unsupported format or MIME. | Reject before inference. |
| C-02 | Image input | Corrupt bytes. | Validation error; |

| | | | |
|---|---|---|---|
| | | | classifier not called. |
| C-03 | Image input | Oversize / invalid dimensions. | Reject with bounded request handling. |
| C-04 | Suitability | Obvious non-leaf image. | Reject or request supported leaf image. |
| C-05 | Classifier | Valid supported crop leaf. | Return top label and score. |
| C-06 | Uncertainty | Score below 0.75. | Return result with uncertain flag. |
| C-07 | Crop consistency | Selected crop conflicts with model crop. | Return uncertain flag. |
| C-08 | Persistence | Successful scan. | Store metadata/hash, not photo. |
| C-09 | Location | Ambiguous town query. | Require selection of intended result. |
| C-10 | Coordinates | Boolean/invalid numeric. | Reject; do not coerce. |
| C-11 | Area | Acres input. | Convert to hectares consistently. |
| C-12 | Soil source | Missing pH/nodata. | Do not invent; prevent unsupported rank. |
| C-13 | Soil cache | Partial response, then recovery. | Do not cache incomplete soil as complete. |
| C-14 | Weather | Missing rainfall. | Do not treat missing as zero. |
| C-15 | Weather | Duplicate or incomplete forecast dates. | Reject inconsistent sequence. |
| C-16 | Weather | Minimum exceeds maximum. | Reject invalid temperature range. |
| C-17 | Network | Transient 502/503/504. | One bounded retry for supported case. |
| C-18 | Quota | Rate-limit response. | No repeated retry loop. |
| C-19 | Planner | Off-window sowing date. | Exclude or explain constraint. |
| C-20 | Planner | pH outside crop interval. | Exclude under pH screen. |
| C-21 | Planner | Historical temp outside crop range. | Exclude or show failed condition. |
| C-22 | Water | Rainfed with dry month. | Apply explicit heuristic. |
| C-23 | Water | Irrigation available. | Show relevant irrigation context. |
| C-24 | Planner | No eligible crop. | Return no-match state. |
| C-25 | Economics | Wheat worked example. | About ₹47,743/ha. |
| C-26 | Frontend | Cancel pending lookup. | Abort and restore controls. |
| C-27 | Frontend | Late response after cancellation. | Ignore stale result. |
| C-28 | Frontend | Backend recovers after outage. | Refresh health status. |
| C-29 | Advisor | Relevant query after migration. | Return passage and citations. |
| C-30 | Advisor | No relevant passage. | Abstain clearly. |
| C-31 | Safety | Chemical dosage query. | Block general dosage advice. |
| C-32 | Database | Missing search_vector field. | Report schema failure; no score. |
| C-33 | Citation | Stored source page/section. | Render stored citation metadata. |
| C-34 | Frontend | Lazy component fails. | Error boundary offers |

| | | | |
|---|---|---|---|
| | | | recovery. |

### C.1 REGRESSION TESTING PRIORITIES

External services can change availability, units or response format. Unit tests should use deterministic fixtures; a separate smoke-test set should exercise live access and label its date. Test parsers against null values, unexpected units, duplicate dates, missing fields, invalid coordinates and truncated raster responses. Cache behavior should prove incomplete values cannot be treated as successful results.

Model scripts should assert identical test indices for baseline, RF and XGBoost. The chronological split must precede tuning; the baseline must use training values only. Record row counts, time boundaries, district overlap and feature encoding with every result. These checks prevent a result from looking better because different models received different test examples or because future information leaked into training.

## APPENDIX D. REPRODUCIBILITY CHECKLIST

### D.1 MOBILE IMAGE MODEL

- Record exact checkpoint path and SHA-256 checksum.
- Store the ten-label class order with the checkpoint.
- Restore image data and split manifest; preserve train/validation/test partitions.
- Record decoder, color channel, resizing, normalization and batch-size choices.
- Record framework versions, inference device, CPU threads and warm-up.
- Export per-class scores, confusion matrix and aggregate metrics.
- Report field-set and calibration results separately.
- Archive command, timestamp, checkpoint version and unedited metrics.

### D.2 RANDOM FOREST AND XGBOOST

- Record the CSV source, retrieval date and checksum.
- Record row filters and supported crop name mapping.
- Record feature list, units, categorical encoding and missing-value policy.

- Record the 1966–2012 training and 2013–2017 test boundaries.
- Record train/test counts and the 304 shared districts.
- Use the exact same held-out rows for every model.
- Tune only within training years or keep parameters fixed and state so.
- Record library versions, hardware and fit/predict timing scope.
- Save per-crop/year residual summaries and model parameters.
- State that offline and UI configurations differ.

### D.3 TEXT ADVISOR

- Apply Alembic migrations and record the database migration head.
- Seed the current six-entry corpus and record source checksums.
- Verify search_vector, query configuration and index.
- Version a query set with human-judged relevant passage IDs.
- Resolve reviewer disagreements and preserve relevance definitions.
- Report precision@k, recall@k, MRR/nDCG and citation validity by topic.
- Include unsupported and dosage prompts for abstention/safety checks.
- Record PostgreSQL version, corpus checksum, query method and latency.
- Archive failed cases without tuning on the final test set.

### D.4 REPORT SCREENSHOT EVIDENCE

Each screenshot should correspond to an evaluation artifact or reproducible UI state. Captions should identify module, date, dataset or input values, model version and whether the numbers were rerun or saved. Keep uncertainty, provisional, benchmark-surplus and experimental labels visible. The metric file and source data manifest should remain available even when screenshots are included.